

Lab 12: GitHub Actions CI/CD for Your Orin Nano Container

Duration: 90 minutes | **Team:** 2 students | **TA validation required by end of today**

Overall Goal

By the end of this lab, your team will have **a public GitHub repository** holding the inference container you built in Lab 10, **a green GitHub Actions workflow** that runs lint + pytest on every push, **a multi-architecture Docker image** built for `linux/arm64` and pushed to **GitHub Container Registry (GHCR)** with the commit SHA as its tag, and **a one-command pull-and-run** on your Orin Nano that confirms the image you just built in the cloud actually works on the device. As a stretch goal (Step 5), teams with time remaining can also register their Jetson as a self-hosted GitHub Actions runner — useful preparation for HW6 but not required for Lab 12 sign-off.

This lab is the hands-on follow-through to Week 10's `cicd-overview` and `github-actions` slides and Week 8's "CI/CD concepts" lecture. The Lab 10 image you'll use is your existing build artifact — the only Dockerfile change today is **deferring the `yolo export format=engine` step to container start** so the build no longer needs `--runtime nvidia`. That refactor is itself a good production pattern (engine compilation is host-specific) and is the single technical move that lets a generic GitHub-hosted x86 runner build an ARM64 Jetson image at all.

About the AI prompts below: Each step includes an *example* prompt to get you started. These are references — you are encouraged to craft your own prompts, experiment with different phrasings, and learn what produces the best results. Good prompt engineering is a skill worth developing.

Reference implementation. Every file the lab walks through is shown verbatim *inside the step that introduces it* — `Dockerfile.ci` builds up across Steps 1 / 3, `.github/workflows/ci.yml` grows job-by-job from Step 2.1 → 3.2, the deferred-engine wrapper `entrypoint.sh` is in Step 3.1, and the smoke-test `tests/test_smoke.py` is in Step 2.2. Each step also embeds the validator it runs (`validate_stepN.sh`). Nothing lives in an appendix; everything appears where it's discussed.

Hardware You Need

- Jetson Orin Nano (original or Super, 8 GB) running JetPack 6 (L4T 36.x) — used in Steps 4 and 6
- Internet connection on both your laptop **and** the Jetson (the workflow runs in GitHub's cloud; the Jetson pulls the resulting image)
- A **public** GitHub repository — every team member needs a GitHub account; GHCR is free for public repos and ARM64 images count against zero quota there
- Your **Lab 10 deliverables** — `Dockerfile.single`, `inference_node.py`, `requirements.txt` from wherever your Lab 10 lives (`~/lab10/`, `~/project/coding/lab10/`, or wherever your team keeps it). Today's CI image inherits all of it.
- Your **HW5 fine-tuned weights** — `best.pt` from your Jetson, where HW5's overnight continued fine-tune produced the ~110-epoch refined version. The weights file is gitignored everywhere (it's a binary artifact, not source) so it's not in any of the lab/HW source folders — it lives only on whichever machine actually trained it. Lab 9's baseline `best.pt` works as a fallback if HW5 isn't done.
- (*Stretch only — for Step 5*) SSH access from your laptop into the Jetson and ~500 MB free disk for the GitHub `actions-runner` install

Why a public repo? GitHub Actions minutes, GHCR storage, and GHCR pulls are all **unmetered for public repositories**. Private repos get 2 000 free Actions minutes/month and 500 MB GHCR storage on the free plan, which is enough for this lab but burns down for HW6. Going public also matches the open-source ethos most edge-AI prototypes ship under. If your Lab 10 code contains anything you don't want public (camera credentials, MQTT passwords, private model weights), put it in `.gitignore` and load it from environment variables — HW6 covers GitHub Secrets for the password case.

Pre-Class Checklist (15 min, do this BEFORE lab session)

The 90 minutes go fast. Show up with these items already done — you'll get to Step 4 instead of Step 1 when class ends.

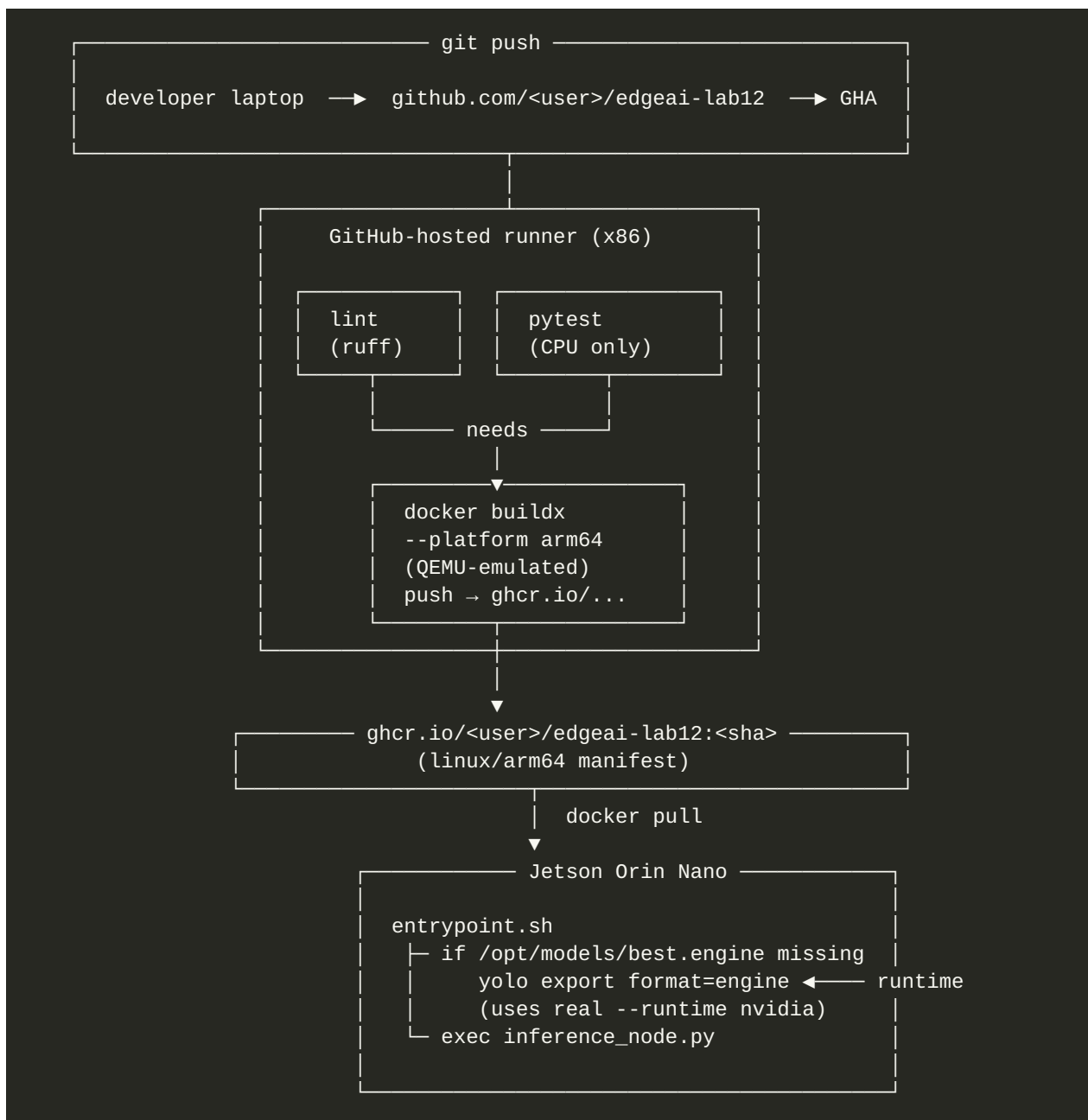
- [] **GitHub account** for each team member, with email verified. (Sign up at <https://github.com/signup>.)
- [] **gh CLI installed and authenticated** on your laptop. Run `gh auth status` and confirm it shows your username.

- [] **gh CLI installed and authenticated on the Jetson** — Step 4 uses `gh auth token | docker login ghcr.io` as the cleanest auth path for the GHCR pull. `sudo apt install gh && gh auth login` on the Jetson; pick the same login flow you used on the laptop. (You can fall back to a Personal Access Token if `gh` is unavailable, but `gh` is simpler.)
- [] **Lab 10 source files on hand** — locate `Dockerfile.single`, `inference_node.py`, and `requirements.txt` wherever your Lab 10 lives (could be `~/lab10/`, `~/project/coding/lab10/`, a GitHub clone, etc.). If they're on a different machine, copy them to your laptop now.
- [] **best.pt from the Jetson** — `best.pt` is gitignored across every Lab/HW (it's a model binary, not source) so it lives only on the machine that trained it. Pre-class, `scp` it from your Orin Nano: `scp <jetson-user>@<jetson-ip>:<path>/best.pt ~/Downloads/`. The `<path>` is wherever HW5 (preferred — ~110-epoch refined) or Lab 9 (baseline fallback) wrote it; common HW5 path is `~/project/coding/hw5/models/best.pt`. To find yours: `ssh <jetson> "find ~ -name best.pt 2>/dev/null"`. Size will be 6–10 MB, well under GitHub's 100 MB single-file limit.
- [] **Disk space on the Jetson:** ≥ 15 GB free in `/var/lib/docker`. `df -h /var/lib/docker | tail -1` — if you're tight, run `docker image prune -a` ahead of class.
- [] **Mosquitto running on the Jetson** at `localhost:1883`. `systemctl status mosquitto`.
- [] **NVIDIA container runtime configured** on the Jetson.
`docker info 2>/dev/null | grep -i nvidia` should show `Runtimes: ... nvidia ...`.
- [] **docker buildx available on the Jetson** — `docker buildx version` should print a version. Step 4's image-architecture check uses it; older Docker installs without buildx will need `sudo apt install docker-buildx-plugin`. (Not strictly required on the laptop — the actual multi-arch build runs inside the GitHub-hosted runner — but useful if you want to dry-parse `Dockerfile.ci` locally per Step 2.3.)
- [] **Pre-pull the dustynv base image on the Jetson** — `docker pull dustynv/pytorch:2.7-r36.4.0`. Step 4's `docker pull` only fetches your application layers if the base is already cached. Saves 5–8 min during lab.
- [] **GitHub Actions enabled** for your account/org. Some education-tier accounts disable Actions by default; check at <https://github.com/settings/actions>.
- [] **Confirm you can create public repos** — some accounts default to private-only. `gh api user --jq .plan` should show a plan that allows public repos.
- [] **One team member is the "repo owner"** — i.e., the GitHub account whose namespace will host the repo. The other can be added as a collaborator (`gh repo edit --add-collaborator <user>`).
- [] (Optional) **Bookmark these URLs** for quick reference during the lab: <https://docs.github.com/actions>, <https://github.com/docker/build-push-action>, <https://docs.github.com/packages/working-with-a-github-packages-registry/working-with-the-container-registry>.

If any item above is unchecked when class starts, do that item first while your teammate moves on — do not block the team on it.

Architecture

You're building two pipelines that share the same Dockerfile. The cloud pipeline runs in GitHub's data center on a hosted x86 runner, builds an ARM64 image via QEMU emulation, and pushes it to GHCR. The on-device pipeline runs on the Jetson, pulls the image, and finishes the GPU-dependent work (TRT engine compile) at container start.



Why split the work this way? The TRT engine compile **must** see a real Jetson GPU — QEMU can emulate ARM64 instructions but cannot present a CUDA device. By moving that step from `RUN` (build time, no GPU) to `entrypoint.sh` (run time on the Jetson, real GPU), the same Dockerfile works for both runners. First boot on the Jetson takes the usual 5–8 min to compile; subsequent boots reuse the cached `.engine` file via a bind-mounted volume.

Why GHCR and not Docker Hub? GHCR auth uses your GitHub token (no extra account), inherits your repo's visibility, and ships free unlimited bandwidth for public images. Docker Hub free tier rate-limits anonymous pulls (100/6h per IP) — fine for a single Jetson but painful when a class of 30 students all pull from the same campus NAT.

How Validation Works

Each step ends with `bash validate_stepN.sh`. These scripts are runnable on either your laptop (Steps 1–3, 5) or your Jetson (Step 4) and **verify wiring across the boundary** — that GitHub sees your push, that the workflow ran, that the image exists in GHCR with the right architecture, that the container starts cleanly on the Jetson, etc. They use the GitHub CLI (`gh`) for repo-side checks because it's the only practical way to query Actions status from a script.

- `gh` is preinstalled on JetPack 6 and on the lab laptops; if missing, `sudo apt install gh` then `gh auth login`.
- **Step 1** is a pre-flight: it runs *before* you push anything and verifies your local repo is set up correctly (`gh` authenticated, remote points to your fork, the Dockerfile parses).
- **Steps 2–3** run after each `git push` and poll GitHub Actions until the latest workflow finishes, then assert green.
- **Step 4** runs on the Jetson and verifies the image you just pushed pulls cleanly, starts, builds the engine on first run, and serves a detection.
- **Step 5** validates that a self-hosted Jetson runner picked up a job and finished it green.

When a check fails, the script prints which subsystem to look at (`gh workflow view`, `docker buildx inspect`, `journalctl -u actions.runner.*`, etc.). Don't move on until the current step is green.

Step 0: Set Up Project Repository (10 min)

Goal

Create a fresh **public** GitHub repo, copy your Lab 10 code into it, install the `gh` CLI if it isn't already there, and verify you can push.

Working folder placement — important

The lab uses `~/lab12` as the example working folder, but the path itself doesn't matter. What does matter: **the working folder must NOT be inside an existing git repository**, regardless of which host that other repo points at (Bitbucket, GitHub, GitLab, etc.). Common gotchas:

- If you keep prior labs in a parent tree like `~/project/coding/` that is itself a git repo (e.g. cloned from Bitbucket), don't `mkdir ~/project/coding/lab12` — you'll create a confusing nested-git situation. Either pick a path outside that tree (`~/lab12`, `~/work/edgeai-lab12`, etc.) or add `lab12/` to the parent repo's `.gitignore` first.
- If your `~/lab10/` is itself inside a git tree, the source-copy commands below are still safe (they only read files), but verify with `git -C ~/lab10 rev-parse --show-toplevel` to know what tree you're in.

When in doubt, run `git rev-parse --show-toplevel` from the proposed working folder *before* `git init`. If it prints a path, you're inside an existing repo — pick somewhere else.

Instructions

```
mkdir -p ~/lab12 && cd ~/lab12

# Copy your Lab 10 source files as the starting point. Replace the LAB10
# variable with wherever your team keeps Lab 10 — examples:
#   LAB10=~/lab10
#   LAB10=~/project/coding/lab10
#   LAB10=~/repos/edgeai-course/lab10
LAB10=~/lab10    # ← edit this one line to match your layout
cp "$LAB10/Dockerfile.single" ./Dockerfile.ci
cp "$LAB10/inference_node.py" .
cp "$LAB10/requirements.txt" .

# best.pt is NOT in source control — it's in .gitignore everywhere
# (every Lab/HW that uses YOLO weights does this so the weights file
# isn't committed by accident). Pull it from your Jetson, where HW5's
# overnight continued fine-tune left a ~110-epoch refined version that
# detects better than Lab 9's baseline. Adjust paths to wherever your
# team keeps its HW5 work — common patterns are shown below.
#
# 1) If HW5 lives at the conventional path:
scp <jetson-user>@<jetson-ip>:~/project/coding/hw5/models/best.pt .
#
```

```
# 2) If you don't remember where it is, find it on the Jetson first:
#   ssh <jetson-user>@<jetson-ip> "find ~ -name best.pt 2>/dev/null"
#   Then scp from whichever path that returns (HW5 preferred over Lab 9).
#
# 3) Last resort fallback – Lab 9's baseline is in:
#   ~/project/coding/lab9/runs/detect/train*/weights/best.pt

# Sanity check
ls -la
```

You should see `Dockerfile.ci`, `inference_node.py`, `requirements.txt`, and `best.pt`. The HW5 weights are typically 6–10 MB (YOLOv8n backbone), well under GitHub's 100 MB single-file limit, so plain Git works without LFS.

Why HW5's `best.pt` instead of Lab 9's bare weights? HW5 ran an overnight continued fine-tune (~110 epochs total) on top of Lab 9's baseline. Same architecture, same filename, but better mAP and more representative of what will run in HW6. If you don't have HW5 weights for any reason, Lab 9's `~/project/coding/lab9/runs/detect/train*/weights/best.pt` works as a fallback.

Why rename to `Dockerfile.ci`? We're going to change the engine-compile step in Step 3 to run at container start instead of build time, and we don't want to break your working Lab 10 build in the process. Two Dockerfiles, two purposes: `Dockerfile.single` stays as the Lab 10 reference, `Dockerfile.ci` is what GitHub Actions builds.

Step 0.1 — Initialize git and connect to GitHub

```
cd ~/lab12
git init -b main
git config user.email "your_team@example.com"
git config user.name "Your Team Name"

cat > .gitignore <<'EOF'
# Python
__pycache__/
*.pyc
.venv/
.pytest_cache/
.coverage
htmlcov/

# PDM (only relevant if you do Step 0.4; harmless to include either way)
.pdm-python
.pdm-build/
__pypackages__/

# Build artifacts
*.engine
*.onnx
```

```
# IDE
.vscode/
.idea/
*.swp
.DS_Store

# Local secrets
.env
EOF

git add .
git commit -m "Initial commit: Lab 10 inference node"
```

Note on `best.pt` — why it's committed here when other Labs/HWs gitignore it. The Lab 9/10/HW5 source repos all gitignore `*.pt` because the model file is a binary build artifact, not source. Lab 12's CI image needs the weights baked in (the `Dockerfile.ci COPY best.pt /opt/models/` line), so for *this* repo specifically we commit it. The HW5-refined `best.pt` is small (6–10 MB), well under GitHub's 100 MB cap. If your team prefers to keep weights out of git long-term, HW6 covers two production patterns: (a) downloading from a release artifact at container start, (b) pulling from a registry alongside the image.

Step 0.2 — Install and authenticate `gh`

```
# If gh is not already installed:
sudo apt update
sudo apt install -y gh

# Authenticate (opens a browser tab; choose HTTPS, paste the device code)
gh auth login

# Confirm:
gh auth status
```

Expected output ends with `Logged in to github.com as <your-username>`. If it doesn't, re-run `gh auth login`.

Step 0.3 — Create the remote repo and push

```
# Create the repo and push in one shot. --public is required for free
# Actions minutes and unlimited GHCR.
gh repo create edgeai-lab12 --public --source=. --remote=origin --push

# Verify
git remote -v
gh repo view --web # opens your browser at the repo URL
```


Result	What You See	What To Do
Pass	Repo opens in browser, files visible, public badge	Proceed to Step 1
repo already exists	gh refuses to create	gh repo delete edgeai-lab12 --yes (if it's yours and empty), then re-run
Permission denied (publickey)	git push fails	gh auth setup-git to install gh as a credential helper
file too large	best.pt > 100 MB (rare for HW5-refined YOLOv8n at 6–10 MB; common only if you accidentally cp'd a YOLOv8x or training-checkpoint file with optimizer state)	Verify size with du -h best.pt ; re-scp the right file. If you genuinely need a >100 MB weights file: git lfs install && git lfs track "*.pt" && git add .gitattributes then re-commit.

Step 0.4 — (Optional) Set up PDM project metadata

If your team has been using **PDM** for Python dependency management since Lab 9 (the course default), set up `pyproject.toml` + `pdm.lock` now so `requirements.txt` stays in sync with a real lockfile. Skip this section if you're using plain `pip` + `requirements.txt` and don't want PDM in the loop — Step 1.2 covers a minimal `pyproject.toml` for ruff/pytest config that works either way.

```
cd ~/lab12

# Install pdm if you don't already have it (your team almost certainly does):
brew install pdm                                # macOS
# or: pip install --user pdm                     # any platform

cat > pyproject.toml <<'EOF'
[project]
name = "edgeai-lab12"
version = "0.1.0"
description = "Lab 12 – GitHub Actions CI/CD for the Lab 10 Orin Nano container"
# Pin to the 3.10 line – the dustynv/l4t-pytorch base image and the Jetson
# system Python are both 3.10. ">=3.10" makes pdm consider 3.11+ wheels
# it later has to skip, emitting many warnings.
requires-python = ">=3.10.*"
dependencies = [
    "ultralytics==8.3.40",
    "opencv-python==4.10.0.84",
    "paho-mqtt==2.1.0",
    "onnxruntime==1.20.0",
    # Pin numpy to the 1.x line – the dustynv container's Jetson torch 2.7.0
    # was compiled against numpy 1.x, and matplotlib (ultralytics transitive)
    # would otherwise pull numpy 2.x and break torch's C bindings at runtime.
    "numpy==1.26.4",
```

```

]

# Dev tools — installed by `pdm install -d`, NOT shipped in the container.
# Used by the CI lint + pytest jobs (Step 1) and the local dev loop.
[dependency-groups]
dev = [
    "ruff>=0.6",
    "pytest>=8",
    "pytest-cov>=5",
]

# These packages are already provided by the dustynv/l4t-pytorch base image
# (Jetson-specific ARM64 + CUDA 12.6 builds). Excluding them from pdm
# resolution keeps `pdm export` from pulling generic x86/CUDA wheels that
# would clobber the working container setup at build time.
[tool.pdm.resolution]
excludes = [
    "torch", "torchvision", "triton", "sympy",
    "cuda-toolkit", "cuda-bindings", "cuda-pathfinder",
    "nvidia-cublas", "nvidia-cuda-cupti", "nvidia-cuda-nvrtc",
    "nvidia-cuda-runtime", "nvidia-cudnn-cu13", "nvidia-cufft",
    "nvidia-cufile", "nvidia-curand", "nvidia-cusolver",
    "nvidia-cusparselt-cu13", "nvidia-nccl-cu13",
    "nvidia-nvjitlink", "nvidia-nvshmem-cu13", "nvidia-nvtx",
]

[build-system]
requires = ["pdm-backend"]
build-backend = "pdm.backend"
EOF

# Resolve, lock, install dev deps. Creates pdm.lock + .venv/.
pdm install -d

# Re-export requirements.txt from the lockfile so Dockerfile.ci stays in sync.
pdm export -f requirements --without-hashes -o requirements.txt

```

Add PDM-specific entries to `.gitignore` so the build/venv noise doesn't get committed:

```

cat >> .gitignore <<'EOF'

# PDM
.pdm-python
.pdm-build/
__pypackages__/
EOF

```

What if my team isn't using PDM? Then skip this section entirely. Step 1.2 will create a minimal `pyproject.toml` with just the ruff/pytest tool config (no `[project]` block, no lockfile). The `requirements.txt` you copied from Lab 10 stays as the source of truth. The downside is no automated lockfile drift detection — if someone bumps a dep in

requirements.txt , you discover the conflict at the next CI build instead of at pdm install time. For a single-week lab this is fine; HW6 and beyond benefit more from PDM.

How to Validate

Save this as `validate_step0.sh` in `~/lab12/` and run it:

```
#!/usr/bin/env bash
# validate_step0.sh – confirm repo exists on GitHub with all four required files
# What this validates:
#   - gh CLI is installed and authenticated
#   - origin remote points to a github.com repo
#   - The repo is public (private repos burn Actions minutes from your quota)
#   - All four required files are present in the latest commit on main
# Exits 0 if all checks pass, 1 otherwise.

set -e
echo "=== Step 0 validator: repo plumbing ==="

# 1. gh installed + authenticated
gh auth status >/dev/null 2>&1 || { echo "FAIL: gh not authenticated. Run 'gh auth login'."; exit 1; }
echo "PASS: gh authenticated"

# 2. origin is on github.com
ORIGIN=$(git remote get-url origin)
[[ "$ORIGIN" == *github.com* ]] || { echo "FAIL: origin is not github.com: $ORIGIN"; exit 1; }
echo "PASS: origin = $ORIGIN"

# 3. Repo is public
VIS=$(gh repo view --json visibility --jq .visibility)
[[ "$VIS" == "PUBLIC" ]] || { echo "FAIL: repo visibility is $VIS, must be PUBLIC"; exit 1; }
echo "PASS: repo is PUBLIC"

# 4. Required files present in the latest commit
for f in Dockerfile.ci inference_node.py requirements.txt best.pt; do
  git cat-file -e "HEAD:$f" 2>/dev/null || { echo "FAIL: $f missing from HEAD"; exit 1; }
  echo "PASS: $f present"
done

# 5. (Optional) If you did Step 0.4, verify pyproject.toml + pdm.lock are committed.
# Skipped silently for non-PDM teams.
for f in pyproject.toml pdm.lock; do
  if [ -f "$f" ]; then
    git cat-file -e "HEAD:$f" 2>/dev/null \
      && echo "PASS: $f present (PDM)" \
      || echo "WARN: $f exists locally but is not committed – git add $f"
  fi
done

echo "=== Step 0 PASS ==="
```

```
chmod +x validate_step0.sh
./validate_step0.sh
```

Expect five `PASS` lines (plus two more if you opted into Step 0.4 PDM setup) and a final `Step 0 PASS`.

Example AI Prompt

"I have a Python + Docker project for Jetson in `~/lab12`. Walk me through creating a public GitHub repo with the `gh` CLI, pushing my initial commit, and writing a bash script that verifies the repo exists, is public, and contains four specific files."

Step 1: Add a Lint + Pytest Workflow (15 min)

Goal

Write your first `.github/workflows/ci.yml` with two jobs — `lint` and `test` — that run on the free `ubuntu-latest` x86 runner. No Docker yet, no Jetson involvement; just confirm the YAML is valid and the workflow runs green on every push.

Step 1.1 — Add a smoke test

Create `tests/test_smoke.py`:

```
"""Lab 12 smoke tests – minimal CPU-only assertions that don't need GPU.

These tests run inside GitHub's free hosted runner (Ubuntu x86), so they
must not import torch with CUDA, must not load TensorRT, and must not
require the IMX219 camera. Their job is to catch the obvious 'I broke
the import graph' or 'the YOLO model file is corrupted' failures BEFORE
the slower Docker build job spends 10 min building an ARM64 image.

Run locally with:    pytest -v tests/
"""

from pathlib import Path

import pytest

def test_best_pt_exists():
    """The fine-tuned weights file must be present in the repo."""
    p = Path(__file__).parent.parent / "best.pt"
    assert p.exists(), f"{p} missing – did you forget to commit best.pt?"
    assert p.stat().st_size > 1_000_000, "best.pt suspiciously small (<1 MB)"

def test_requirements_pinned():
    """Every dep in requirements.txt should be pinned (== or ~=)."""
    req = (Path(__file__).parent.parent / "requirements.txt").read_text()
```

```

for line in req.splitlines():
    line = line.strip()
    if not line or line.startswith("#"):
        continue
    # Allow editable installs and -e/-r/-c directives
    if line.startswith((" -e", "-r", "-c", "--")):
        continue
    assert "==" in line or "~=" in line or ">=" in line, \
        f"Unpinned dep in requirements.txt: {line!r}"

def test_dockerfile_uses_arm64_base():
    """Dockerfile.ci must use a Jetson-compatible (ARM64) base image."""
    df = (Path(__file__).parent.parent / "Dockerfile.ci").read_text()
    # dustynv images are aarch64-only; l4t-base is also aarch64-only.
    # Either is acceptable; a generic python:3.11 base would silently
    # build for x86 and never run on the Jetson.
    assert any(base in df for base in ["dustynv/", "l4t-", "nvcr.io/nvidia/l4t"]), \
        "Dockerfile.ci must FROM a Jetson ARM64 base (dustynv/* or l4t-*)"

@pytest.mark.parametrize("name", ["inference_node.py", "best.pt", "requirements.txt"])
def test_required_files(name):
    """Files the Docker COPY steps reference must exist."""
    assert (Path(__file__).parent.parent / name).exists(), f"{name} missing"

```

Why no test for the build-time engine compile yet? The Lab 10 `Dockerfile.single` you copied to `Dockerfile.ci` *does* contain a build-time engine compile — that's the whole thing Step 2 fixes. Adding a smoke test for "no `format=engine` in any RUN line" here would guarantee a red CI on first push, defeating the point of getting a green Step 1 baseline. Step 2's `validate_step2.sh` script explicitly checks the same property after the refactor lands; that's where the assertion belongs.

```

mkdir -p tests
# (paste the test_smoke.py content above into tests/test_smoke.py)

```

Step 1.2 — Add a ruff config

Create `pyproject.toml` (a minimal one — the Lab 10 project may already have one with PDM config; either add to it or create fresh):

```

[tool.ruff]
line-length = 100
target-version = "py310"

[tool.ruff.lint]
# Pick a small starter set so the first run isn't a wall of red.
select = ["E", "F", "I", "W"]
ignore = ["E501"] # line-too-long: leave for the formatter to handle

```

```
[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-v --tb=short"
```

Step 1.3 — Write the workflow

Create `.github/workflows/ci.yml`:

```
# Lab 12 CI workflow – runs on every push and PR to any branch.
# Two jobs:
# 1. lint – ruff check (style + import order + obvious bugs)
# 2. test – pytest on the smoke tests in tests/
# Both run on the free GitHub-hosted ubuntu-latest x86 runner.
# The Docker build (Step 3) and on-device pull (Step 4) come later.

name: CI

on:
  push:
    branches: ["**"] # any branch
  pull_request:
    branches: [main]

jobs:
  lint:
    name: Ruff lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5

      - name: Set up Python
        uses: actions/setup-python@v6
        with:
          python-version: "3.10"
          cache: pip

      - name: Install ruff
        run: pip install ruff==0.6.9

      - name: Run ruff
        # --output-format=github makes errors show up as inline annotations
        # on the PR diff, not just in the log.
        run: ruff check . --output-format=github

  test:
    name: Pytest smoke
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5

      - name: Set up Python
        uses: actions/setup-python@v6
        with:
          python-version: "3.10"
          cache: pip
```

```

- name: Install test deps
  # We deliberately don't pip-install requirements.txt here — most of
  # those packages need a Jetson + CUDA wheel that doesn't exist on
  # x86. The smoke tests only need pytest itself.
  run: pip install pytest==8.3.3

- name: Run pytest
  run: pytest tests/ -v

```

```

mkdir -p .github/workflows
# (paste the ci.yml content above into .github/workflows/ci.yml)

```

Step 1.4 — Commit and push

```

git add tests/ pyproject.toml .github/
git commit -m "Add lint + pytest workflow"
git push origin main

```

Open the repo in your browser (`gh repo view --web`) and click **Actions** in the top nav. You should see your "CI" workflow running.

Heads-up on `gh run watch`. If you try `gh run watch` immediately after `git push`, you may see "found no in progress runs to watch" — Step 1's lint+test jobs often complete in 15–30 s, faster than the polling cycle. That's fine; check the result with `gh run list --workflow=ci.yml --limit 1` instead. `gh run watch` is more useful in Step 3, where the build job runs for 20–40 minutes.

What you'll see in the Actions UI (right after the push):

```

□ CI                                on main • <commit-sha>    0s
  ▶ □ Ruff lint                      queued
  ▶ □ Pytest smoke                   queued

```

After ~10 s the runners start (status changes from `queued` → `in progress`):

```

□ CI                                on main • <commit-sha>    20s
  ▶ □ Ruff lint                      in progress
  ▶ □ Pytest smoke                   in progress

```

The two jobs run in parallel and finish in ~25–40 s each:

□ CI	on main • <commit-sha>	45s
▸ □ Ruff lint		35s
▸ □ Pytest smoke		28s

Click into either job to see the full log — useful for debugging the next time something fails. The "Run pytest" step inside the test job should print:

```
===== test session starts =====
collected 6 items

tests/test_smoke.py::test_best_pt_exists PASSED [ 16%]
tests/test_smoke.py::test_requirements_pinned PASSED [ 33%]
tests/test_smoke.py::test_dockerfile_uses_arm64_base PASSED [ 50%]
tests/test_smoke.py::test_required_files[inference_node.py] PASSED [ 66%]
tests/test_smoke.py::test_required_files[best.pt] PASSED [ 83%]
tests/test_smoke.py::test_required_files[requirements.txt] PASSED [100%]

===== 6 passed in 0.12s =====
```

Why we don't `pip install -r requirements.txt` in CI: the Jetson-specific PyTorch wheel doesn't exist on PyPI for x86 (it's only published for `aarch64+cuda` via the `dustynv` mirror). Trying to install it on the GitHub-hosted Ubuntu x86 runner fails with `Could not find a version that satisfies the requirement torch==2.5.x.nv24...`. Smoke tests deliberately don't import torch — they validate file presence, not runtime behavior. Real torch tests happen in the `integration-test` job in HW6, which runs on the Jetson.

If lint fails: `ruff check . --fix` locally, commit the auto-fixes, push again. If test fails: `pytest tests/ -v` locally and read the assertion message — usually `best.pt` wasn't actually copied into the working folder, or `git add .` was run before `best.pt` landed there. Verify with `git ls-files | grep best.pt`.

How to Validate

Save as `validate_step1.sh`:

```
#!/usr/bin/env bash
# validate_step1.sh — verify the latest workflow run on main is green.
# Polls every 5 s for up to 3 min; fails if the run is failed/cancelled
# or if no run has been triggered.

set -e
echo "=== Step 1 validator: workflow green ==="

REPO=$(gh repo view --json nameWithOwner --jq .nameWithOwner)
echo "Repo: $REPO"

# Wait for the latest run to finish (max 3 min)
```



```

for i in {1..36}; do
  STATUS=$(gh run list --workflow=ci.yml --limit 1 --json status,conclusion --jq '.[0]')
  S=$(echo "$STATUS" | jq -r .status)
  C=$(echo "$STATUS" | jq -r .conclusion)

  if [ "$S" = "completed" ]; then
    if [ "$C" = "success" ]; then
      echo "PASS: workflow conclusion = success"
      echo "=== Step 1 PASS ==="
      exit 0
    else
      echo "FAIL: workflow conclusion = $C"
      echo "  Open in browser: gh run view --web"
      exit 1
    fi
  fi

  echo "  attempt $i/36: status=$S, waiting 5 s..."
  sleep 5
done

echo "FAIL: workflow did not complete within 3 min"
exit 1

```

```

chmod +x validate_step1.sh
./validate_step1.sh

```

Expected output:

```

=== Step 1 validator: workflow green ===
Repo: your-user/edgeai-lab12
  attempt 1/36: status=in_progress, waiting 5 s...
  attempt 2/36: status=in_progress, waiting 5 s...
  attempt 3/36: status=in_progress, waiting 5 s...
PASS: workflow conclusion = success
=== Step 1 PASS ===

```

Step 1 Troubleshooting

Symptom	Likely cause	Fix
Workflow doesn't trigger after <code>git push</code>	<code>.github/workflows/ci.yml</code> is on a branch GitHub Actions doesn't watch (e.g., you pushed to <code>dev</code> and the trigger only listens to <code>main</code>)	The <code>on: push: branches: ["**"]</code> block triggers any branch — confirm that's what your file says

Symptom	Likely cause	Fix
Lint job fails with error: Failed to parse pyproject.toml	Indentation in TOML (use spaces, not tabs); or a stray quote	<code>python -m tomllib < pyproject.toml</code> locally to find the offending line
Lint job fails with hundreds of errors on first run	Ruff's default rule set is broader than the lab's <code>select = ["E", "F", "I", "W"]</code>	Check your <code>pyproject.toml</code> 's <code>[tool.ruff.lint]</code> section uses the same <code>select</code> list as Step 1.2
Test job fails with <code>ModuleNotFoundError: No module named 'pytest'</code>	Workflow forgot the <code>pip install pytest</code> step	Compare your <code>ci.yml</code> with Step 1.3; the Install test deps step is required
Test job fails with <code>best.pt</code> missing – did you forget to commit <code>best.pt</code> ?	<code>best.pt</code> not actually in working folder, or got skipped by <code>git add .</code> (e.g. you have <code>*.pt</code> in <code>.gitignore</code> from a Lab 10 carry-over)	Verify locally: <code>ls -la best.pt && git ls-files \ grep best.pt</code> . If it's there but not tracked, check <code>.gitignore</code> for stray <code>*.pt</code> patterns and remove them. If genuinely >100 MB (rare for HW5-refined): <code>git lfs install && git lfs track "*.pt" && git add .gitattributes best.pt && git commit && git push</code> .
Test passes locally but fails in CI	Pytest version mismatch; CI runs 8.3.3 (pinned in <code>ci.yml</code>)	Pin the same version locally: <code>pip install pytest==8.3.3</code>
Workflow stuck on "queued" >2 min	GitHub Actions free- tier capacity hiccup (rare)	Wait 5 min; if still queued, cancel and re- run via <code>gh run rerun</code>

Example AI Prompt

"Write a GitHub Actions workflow for a Python project. It should have two jobs that run on every push: (1) ruff lint with rule set [E,F,I,W], (2) pytest on a `tests/` directory. Use Python 3.10, pin ruff and pytest to specific versions, surface lint errors as inline PR annotations. Don't pip-install the project's main requirements.txt — those wheels only exist for aarch64+CUDA. Then write a bash script that uses the gh CLI to wait for the latest run on main to finish and exits 0 only if it succeeded."

Step 2: Refactor for Build-Time GPU Independence (15 min)

Goal

Modify your Lab 10 Dockerfile so it **does not need a GPU at build time**. The two GPU-dependent steps from Lab 10 — `pip install` of GPU wheels and the `yolo export format=engine` — both move out: `pip` swaps to CPU-only ONNX runtime (already done in Lab 10's `Dockerfile.single`) and the engine compile moves to the container's `entrypoint.sh`, where it runs on the Jetson with real `--runtime nvidia`.

Step 2.1 — Write the entrypoint script

Create `entrypoint.sh`:

```
#!/usr/bin/env bash
# entrypoint.sh – runs at container start on the Jetson.
#
# What it does:
# 1. If best.engine is missing or older than best.pt, compile it with
#    Ultralytics' yolo export. This needs the GPU (--runtime nvidia)
#    and uses the container's TensorRT 10.3.
# 2. exec inference_node.py so signals (SIGTERM from `docker stop`)
#    are delivered to the Python process, not to bash.
#
# Why deferred to runtime instead of `RUN` in the Dockerfile?
# QEMU emulation on the GitHub-hosted x86 runner can fake aarch64
# instructions but cannot present a CUDA device, so `yolo export
# format=engine` would fail at build time. Running it at container
# start on the Jetson sidesteps the issue and matches production
# practice – engines are host-specific anyway (driver/TRT version
# on the build host must match the run host, so binding the compile
# to first run guarantees they match).
#
# First run takes 5–8 min. Mount /opt/models as a volume so subsequent
# runs reuse the cached engine:
# docker run -v lab12-models:/opt/models ...

set -euo pipefail

MODEL_DIR=/opt/models
WEIGHTS=${MODEL_DIR}/best.pt
ENGINE=${MODEL_DIR}/best.engine

if [ ! -f "${WEIGHTS}" ]; then
    echo "ERROR: ${WEIGHTS} not found. Did you copy best.pt into the image?" >&2
    exit 1
fi

# Recompile if engine is missing or older than weights.
# The cd is in a subshell so it cannot leak into the exec below –
# the parent shell stays in /app (the Dockerfile's WORKDIR), which
# is where inference_node.py lives.
```

```

if [ ! -f "${ENGINE}" ] || [ "${WEIGHTS}" -nt "${ENGINE}" ]; then
    echo "[entrypoint] Compiling TensorRT engine (this takes 5-8 min on first boot)..."
    (
        cd "${MODEL_DIR}"
        python3 -c "
from ultralytics import YOLO
YOLO('best.pt', task='detect').export(format='engine', imgsz=320, half=True, opset=19)
"
    )
    echo "[entrypoint] Engine compiled: $(ls -lh ${ENGINE} | awk '{print $5}')"
else
    echo "[entrypoint] Reusing cached engine: $(ls -lh ${ENGINE} | awk '{print $5}')"
fi

# Run whatever CMD the Dockerfile specifies (default: python3 inference_node.py).
# `exec` replaces bash with that process so SIGTERM from `docker stop`
# reaches it directly and graceful-shutdown handlers actually run.
exec "$@"

```

```
chmod +x entrypoint.sh
```

Step 2.2 — Modify the Dockerfile

Open `Dockerfile.ci` and replace the engine-compile RUN with a COPY of `entrypoint.sh` plus an ENTRYPOINT directive. Concretely, change the bottom of the file from this:

```

# OLD — Lab 10 single-stage Dockerfile, last RUN block:
COPY inference_node.py .
RUN mkdir -p /opt/models
COPY best.pt /opt/models/

# Compile the TensorRT engine inside the container so it matches the
# container's TRT version (10.3), not the host's (10.7). First build takes
# 5-8 min; result is cached in this image layer for all future builds.
RUN cd /opt/models && \
    python3 -c "from ultralytics import YOLO; \
        YOLO('best.pt', task='detect').export(format='engine', imgsz=320, half=True, opse

```

...to this:

```

# NEW — Dockerfile.ci, deferred engine compile
COPY inference_node.py /app/
COPY entrypoint.sh /app/
RUN chmod +x /app/entrypoint.sh && mkdir -p /opt/models
COPY best.pt /opt/models/

# No GPU work in this image. The TRT engine is compiled at container
# start by entrypoint.sh — see that file for rationale.
WORKDIR /app
ENTRYPOINT ["/app/entrypoint.sh"]

```

Step 2.3 — Smoke-test the Dockerfile parses

Even without a Docker daemon you can ask `docker buildx` to dry-parse:

```
# (Skip this if you don't have docker on the laptop – it's optional.)
docker buildx build --platform linux/arm64 --no-cache -f Dockerfile.ci . --check
```

If `docker` isn't installed locally, skip — the real build happens in CI in Step 3.

How to Validate

Save as `validate_step2.sh`:

```
#!/usr/bin/env bash
# validate_step2.sh – verify the Dockerfile no longer contains a build-time
# yolo export, and that entrypoint.sh exists with the right shebang.
# Exits 0 if all checks pass, 1 otherwise.

set -e
echo "=== Step 2 validator: Dockerfile refactored ==="

# 1. entrypoint.sh exists and is executable
[ -x entrypoint.sh ] || { echo "FAIL: entrypoint.sh missing or not executable"; exit 1; }
echo "PASS: entrypoint.sh present + executable"

# 2. entrypoint has the required shebang
head -1 entrypoint.sh | grep -q '^#!/usr/bin/env bash' \
|| { echo "FAIL: entrypoint.sh missing shebang"; exit 1; }
echo "PASS: shebang OK"

# 3. Dockerfile.ci has an ENTRYPOINT pointing at it
grep -q 'ENTRYPOINT.*entrypoint.sh' Dockerfile.ci \
|| { echo "FAIL: Dockerfile.ci does not ENTRYPOINT entrypoint.sh"; exit 1; }
echo "PASS: Dockerfile.ci wires ENTRYPOINT"

# 4. Dockerfile no longer has a build-time engine compile.
# A bare grep (no -q) so we can show the offending line in the FAIL message.
if BAD=$(grep -nE "RUN.*format=['\"]?engine['\"]?" Dockerfile.ci); then
    echo "FAIL: Dockerfile.ci still has a RUN line that compiles the engine:"
    echo "  $BAD"
    echo "  Move that step into entrypoint.sh per Step 2.1."
    exit 1
fi
echo "PASS: no build-time engine compile in Dockerfile.ci"

echo "=== Step 2 PASS ==="
```

```
chmod +x validate_step2.sh
./validate_step2.sh
git add entrypoint.sh Dockerfile.ci validate_step2.sh
git commit -m "Defer TRT engine compile to container entrypoint"
git push
```

Expected output:

```
=== Step 2 validator: Dockerfile refactored ===
PASS: entrypoint.sh present + executable
PASS: shebang OK
PASS: Dockerfile.ci wires ENTRYPOINT
PASS: no build-time engine compile in Dockerfile.ci
=== Step 2 PASS ===
```

The push triggers another CI run — lint and pytest should still be green (the smoke tests didn't change). The Step 2 refactor itself is verified by `validate_step2.sh` above; the green CI just confirms nothing else broke.

Step 2 Troubleshooting

Symptom	Likely cause	Fix
<code>validate_step2.sh</code> says <code>entrypoint.sh</code> missing or not executable	Forgot <code>chmod +x entrypoint.sh</code> after creating the file	<code>chmod +x entrypoint.sh</code> and re-commit
Git on Windows or macOS strips the executable bit on push	Filesystem doesn't preserve mode	Use <code>git update-index --chmod=+x entrypoint.sh</code> (one-time fix that records the mode in the index)
<code>Dockerfile.ci</code> does not <code>ENTRYPOINT entrypoint.sh</code>	Used <code>CMD ["entrypoint.sh"]</code> instead of <code>ENTRYPOINT</code>	The two are different — <code>CMD</code> is overridable by <code>docker run image other-command</code> , <code>ENTRYPOINT</code> always runs. Use <code>ENTRYPOINT</code> .
Container starts but immediately exits with <code>bash: /app/entrypoint.sh: Permission denied</code>	Image was built before the <code>chmod</code> fix	Rebuild and re-push (<code>git commit --allow-empty -m "rebuild"; git push</code>)
Container starts but exits with <code>exec /app/entrypoint.sh: no such file or directory</code>	Shebang is <code>#!/bin/bash</code> but the image's bash lives at <code>/usr/bin/bash</code> (Debian quirk on some bases)	The recommended <code>#!/usr/bin/env bash</code> works on every distro because it lets <code>env</code> find bash via <code>\$PATH</code>
First-run engine compile takes >15 min	The container ran out of GPU memory mid-compile (Jetson 8 GB shared)	<code>sudo systemctl stop nvargus-daemon</code> , kill any other CUDA processes, retry

Symptom	Likely cause	Fix
Engine recompiles every run despite the volume mount	The volume isn't mounted at the path <code>entrypoint.sh</code> writes to	Confirm <code>-v lab12-models:/opt/models</code> matches the <code>MODEL_DIR=/opt/models</code> line in <code>entrypoint.sh</code>

Why we use `exec` at the end of `entrypoint.sh`: without `exec`, the bash process becomes the parent of the python process. When `docker stop` sends SIGTERM, it goes to bash (PID 1 in the container). Bash doesn't forward signals by default, so Python never sees the SIGTERM and gets killed by SIGKILL after Docker's 10-second grace period — no graceful shutdown, possible MQTT message loss. With `exec`, Python *replaces* bash as PID 1 and gets SIGTERM directly. This is the same pattern Lab 10 used in `Dockerfile.single`'s last command — Lab 12 just makes it explicit at the `entrypoint` level.

Example AI Prompt

"My Dockerfile compiles a TensorRT engine with `RUN python3 -c 'YOLO(\"best.pt\").export(format=\"engine\")'`. This needs `--runtime nvidia` which I can't get on a generic GitHub Actions runner. Help me move that step into a bash `entrypoint` script that runs at container start instead, with a check to skip recompilation if the engine is newer than the weights. Also explain why the script should `exec python3` instead of just running it directly."

Step 3: Add the Multi-Arch Docker Build Job (5 min active + 20–40 min build wait)

Goal

Add a third job to `ci.yml` that runs **after** lint+test pass, builds `Dockerfile.ci` for `linux/arm64` using `docker buildx` + QEMU emulation, and pushes the resulting image to `ghcr.io/<your-username>/edgeai-lab12:<commit-sha>` plus `:latest`.

Step 3.1 — Understand the moving parts

Three GitHub-published actions do all the work:

Action	What it does
<code>docker/setup-qemu-action@v4</code>	Registers QEMU as a binfmt handler so the x86 kernel can run aarch64 binaries during the build (this is what makes cross-arch builds possible from a single hosted runner).
<code>docker/setup-buildx-action@v4</code>	Installs Docker Buildx — a frontend that knows how to drive multi-platform builds via BuildKit. The default Docker builder can't do <code>--platform</code> cross-compiles.
<code>docker/build-push-action@v7</code>	Wraps <code>docker buildx build</code> with cache-from / cache-to handling, automatic GHCR auth via the workflow's <code>GITHUB_TOKEN</code> , and tag generation.

You also need to **grant the workflow** `packages: write` **permission**, because the default `GITHUB_TOKEN` is read-only for packages.

Step 3.2 — Extend the workflow

Open `.github/workflows/ci.yml` and add this block at the bottom (under the existing `test` job):

```
build:
  name: Build & push ARM64 image
  runs-on: ubuntu-latest
  needs: [lint, test]    # only build if lint + test pass
  permissions:
    contents: read
    packages: write      # required for ghcr.io push
  steps:
    - uses: actions/checkout@v5

    - name: Set up QEMU
      # Lets the x86 runner execute aarch64 binaries inside the build.
      uses: docker/setup-qemu-action@v4
      with:
        platforms: arm64

    - name: Set up Buildx
      uses: docker/setup-buildx-action@v4

    - name: Log in to GHCR
      # GITHUB_TOKEN is auto-provided by the workflow runner; it's
      # scoped to this repo and is what the `packages: write` grant
      # above unlocks.
      uses: docker/login-action@v4
      with:
        registry: ghcr.io
        username: ${ github.actor }
        password: ${ secrets.GITHUB_TOKEN }

    - name: Compute image tags
```



```

id: meta
# Tag every push with the short SHA, and additionally tag pushes
# to main with :latest. PR builds get only :pr-<number>.
# Note: `images:` is just `${{ github.repository }}` (which is
# already `<user>/<repo>`) — adding `/edgeai-lab12` would
# path-double to `<user>/edgeai-lab12/edgeai-lab12`.
uses: docker/metadata-action@v6
with:
  images: ghcr.io/${{ github.repository }}
  tags: |
    type=sha,format=short
    type=ref,event=branch
    type=ref,event=pr
    type=raw,value=latest,enable={{is_default_branch}}

- name: Build and push
  uses: docker/build-push-action@v7
  with:
    context: .
    file: Dockerfile.ci
    platforms: linux/arm64
    push: true
    tags: ${{ steps.meta.outputs.tags }}
    labels: ${{ steps.meta.outputs.labels }}
    # Speed up subsequent builds by caching layers in GHA's cache.
    cache-from: type=gha
    cache-to: type=gha,mode=max

```

Step 3.3 — Watch out for the build time

QEMU-emulated ARM64 builds are **5–10× slower** than native. Lab 10's `Dockerfile.single` builds in ~6 min on a Jetson; the same image under emulation on `ubuntu-latest` typically takes **20–40 min** (longer on a cold first build with the full Jetson torch wheel; faster with PDM resolution-excludes and warm cache). Three things keep this manageable:

1. The `cache-from: type=gha / cache-to: type=gha,mode=max` lines mean the *first* build is slow but every push that doesn't change `requirements.txt` reuses the cached pip layer (~25 min saved).
2. The `needs: [lint, test]` gate skips the build entirely on broken commits.
3. Step 5 of this lab (stretch) registers your Jetson as a self-hosted runner, dropping the build to ~6 min for the next push. HW6 builds on the same setup, so doing Step 5 now pays back twice.

For this lab, **expect your first push to take 20–40 min depending on cache state and how lean your `requirements.txt` is**. Teams who did Step 0.4 PDM setup (with `[tool.pdm.resolution].excludes` keeping torch/CUDA wheels out of the export) typically land at the low end (~20 min); teams whose `requirements.txt` includes the full Jetson torch stack will run closer to 40 min. The active work in this step is only ~5 minutes (the YAML edit + push); the rest is wall-clock waiting for the cloud build.

What to do during the build wait (a productive 40 min, not a coffee break):

1. **Open the build job's live logs in another browser tab** (`gh run view --web`) — watching the buildx output makes the wait less painful and helps you spot failures early
2. **On the Jetson, pre-pull the dustynv base image** so Step 4's `docker pull` only fetches your application layers. Saves 5–8 min later:

```
docker pull dustynv/pytorch:2.7-r36.4.0
```

3. **Read ahead through Step 4** so you're ready to execute the moment the build completes
4. **Verify Step 4 prerequisites on the Jetson** — `docker info | grep -i nvidia`, `df -h /var/lib/docker | tail -1`, `gh auth status`
5. **Take a 5-min break.** Forty minutes is genuinely a long time to stare at a progress bar.

Step 3.4 — Push and watch

```
git add .github/workflows/ci.yml
git commit -m "Add multi-arch docker build + GHCR push"
git push
```

Open Actions in the browser. You should see three jobs: `Ruff lint`, `Pytest smoke`, and `Build & push ARM64 image`. The build job spends most of its time on the `Build and push` step — click into it to see live logs streaming from inside the buildx container.

What you'll see in the build logs (highlights, paraphrased):

```
Run docker/setup-qemu-action@v4
  Setting up QEMU for arm64
Run docker/setup-buildx-action@v4
  Creating a new builder instance
Run docker/login-action@v4
  Logging into ghcr.io
Run docker/metadata-action@v6
  Tags: ghcr.io/your-user/edgeai-lab12:sha-abc1234, ghcr.io/.../edgeai-lab12:latest
Run docker/build-push-action@v7
  #1 [internal] load .dockerignore
  #2 [internal] load build definition from Dockerfile.ci
  #3 [internal] load metadata for docker.io/dustynv/pytorch:2.7-r36.4.0
  #4 [internal] load build context
  #5 [stage-0 1/8] FROM dustynv/pytorch:2.7-r36.4.0 ← 7 GB pull, ~3 min
  #6 [stage-0 2/8] RUN apt-get update && apt-get install -y libgl1-mesa-glx libglu1-mesa-dev
  #7 [stage-0 3/8] WORKDIR /app
  #8 [stage-0 4/8] COPY requirements.txt .
  #9 [stage-0 5/8] RUN pip install --no-cache-dir -r requirements.txt ← 25-35 min under QEMU
  #10 [stage-0 6/8] RUN ORT_VER=... ← stub onnxruntime-gpu
```

```
#11 [stage-0 7/8] COPY inference_node.py /app/
#12 [stage-0 8/8] COPY entrypoint.sh /app/
#13 [stage-0 9/8] RUN chmod +x /app/entrypoint.sh && mkdir -p /opt/models
#14 [stage-0 10/8] COPY best.pt /opt/models/
#15 exporting to image
#16 pushing ghcr.io/your-user/edgeai-lab12:sha-abc1234
✓ pushed in 2.3s
```

When it finishes successfully, go to your repo's home page → right sidebar → **Packages** → `edgeai-lab12` . Click in: you should see a `linux/arm64` manifest tagged with both your short SHA and `latest` .

What the GHCR package page should show: - **Visibility:** Public (or Private — both work, public skips the docker login on the Jetson) - **OS / Arch:** linux/arm64 (this is the moment you confirm cross-compile worked) - **Tags:** at minimum `sha-<7chars>` and `latest` - **Pulls:** 0 (no one has pulled it yet — Step 4 is when you fix that) - **Last published:** just now

How to Validate

Save as `validate_step3.sh` :

```
#!/usr/bin/env bash
# validate_step3.sh - verify the most recent workflow run produced a
# linux/arm64 image in GHCR with the current commit's short SHA tag.

set -e
echo "=== Step 3 validator: ARM64 image in GHCR ==="

REPO=$(gh repo view --json nameWithOwner --jq .nameWithOwner)
SHA=$(git rev-parse --short HEAD)
OWNER=$(echo "$REPO" | cut -d/ -f1)
# Package name in GHCR == the repo name (since the workflow's
# `images: ghcr.io/${{ github.repository }}` line uses just <owner>/<repo>).
PKG=$(echo "$REPO" | cut -d/ -f2)
IMAGE="ghcr.io/$REPO"

echo "Looking for $IMAGE:sha-$SHA"

# 1. Wait for the build job to finish (up to 50 min - QEMU is slow)
for i in {1..600}; do
  STATUS=$(gh run list --workflow=ci.yml --limit 1 --json status,conclusion --jq '.[0]')
  S=$(echo "$STATUS" | jq -r .status)
  C=$(echo "$STATUS" | jq -r .conclusion)
  if [ "$S" = "completed" ]; then
    [ "$C" = "success" ] || { echo "FAIL: workflow conclusion=$C"; exit 1; }
    echo "PASS: workflow completed successfully"
    break
  fi
  [ $((i % 12)) -eq 0 ] && echo " ...$((i*5/60)) min elapsed, status=$S"
  sleep 5
done

# 2. Check the package exists in GHCR via the GitHub API. The endpoint
```

```

# differs for personal accounts vs. organisation accounts, so detect
# the owner type first via gh's /users/{owner} → .type field.
OWNER_TYPE=$(gh api "/users/$OWNER" --jq .type 2>/dev/null || echo "User")
if [ "$OWNER_TYPE" = "Organization" ]; then
    PKG_PATH="/orgs/$OWNER/packages/container/$PKG/versions"
else
    PKG_PATH="/users/$OWNER/packages/container/$PKG/versions"
fi

gh api "$PKG_PATH" --jq '[0].metadata.container.tags' \
> /tmp/tags.json 2>/dev/null \
|| { echo "FAIL: cannot read GHCR package metadata at $PKG_PATH. Did the build push succeed?"; }

if grep -q "sha-$SHA" /tmp/tags.json; then
    echo "PASS: GHCR has tag sha-$SHA"
else
    echo "FAIL: no sha-$SHA tag found. Got: $(cat /tmp/tags.json)"
    exit 1
fi

# 3. Verify the manifest reports linux/arm64. `docker manifest inspect`
# yields either a manifest *list* (multi-arch, fields under .manifests[])
# or a single manifest (.architecture at the top level). Inspect once,
# extract whatever arch fields are present, and look for arm64.
if ! MANIFEST=$(docker manifest inspect "$IMAGE:sha-$SHA" 2>&1); then
    echo "WARN: docker not installed or manifest inspect failed —"
    echo "  verify manually in the GHCR web UI: https://github.com/$REPO/pkgs/container/$PKG"
else
    ARCHES=$(echo "$MANIFEST" | jq -r '.manifests[]?.platform.architecture, .architecture' 2>/dev/null)
    if echo "$ARCHES" | grep -qE '^(arm64|aarch64)$'; then
        echo "PASS: manifest reports arm64"
    else
        echo "FAIL: manifest is not arm64 (saw: $ARCHES)"
        exit 1
    fi
fi

echo "=== Step 3 PASS ==="

```

```

chmod +x validate_step3.sh
./validate_step3.sh

```

Heads-up on `docker manifest inspect` : if your laptop doesn't have Docker, `docker manifest inspect` won't exist either. In that case, just verify the tag in the GHCR web UI (your repo → Packages → `edgeai-lab12` → click the tag). The first two checks (workflow green, GHCR tag exists) are sufficient for TA validation.

Step 3 Troubleshooting

Symptom	Likely cause	Fix
Build job fails immediately with <code>unauthorized: authentication required</code>	Workflow missing the <code>permissions: packages: write</code> block	Re-read Step 3.2 — the <code>permissions:</code> block goes either at the top of the file (file-wide) or inside the <code>build</code> job (job-scoped). Either works; one of them must be present.
Build job fails with <code>failed to solve: dustynv/pytorch:2.7-r36.4.0: failed to copy: read tcp i/o timeout</code>	Docker Hub temporarily slow / rate-limited the dustynv layers	Re-run via <code>gh run rerun --failed</code> . If persistent, mirror the base into your own GHCR ahead of time.
Build hangs at <code>RUN pip install</code> for >40 min with no progress	Standard QEMU slowdown on the ultralytics + opencv install	Wait. Subsequent builds with cache hit drop this to <30 s.
Build succeeds but no GHCR package appears	The <code>tags: template</code> produced an empty list (e.g. on PR builds with <code>enable={{is_default_branch}}</code> only)	Add a <code>type=ref,event=pr</code> tag template so PR builds also get a tag
GHCR package exists but tagged only with <code>pr-N</code> , not <code>sha-XXX</code>	<code>docker/metadata-action</code> 's <code>type=sha,format=short</code> line is missing	Compare your <code>meta</code> step with Step 3.2
Image pushes but architecture shows <code>amd64</code> not <code>arm64</code>	<code>platforms: linux/arm64</code> was forgotten in <code>build-push-action</code>	The default platform is the runner's (x86), so omitting <code>platforms:</code> produces an x86 image
Build job's logs say <code>cache exported</code> but next build still slow	Cache is scoped per ref by default — your <code>main</code> cache doesn't help PRs from feature branches	Use <code>cache-to:</code> <code>type=gha,mode=max,scope=\$ref</code> (mode=max already enabled here)

Symptom	Likely cause	Fix
Workflow burns ~50 GHA minutes per run on free tier	QEMU is slow but not your problem yet — public repos have unlimited minutes	If you went private: bonus #4 in HW6 (self-hosted Jetson runner) is your escape hatch — drops build to ~6 min native

Example AI Prompt

"I have a GitHub Actions workflow with lint and test jobs. Add a third job that builds a linux/arm64 Docker image from `Dockerfile.ci` using `buildx + QEMU` and pushes it to GHCR (`ghcr.io/<repo>/edgeai-lab12`). Tag with the short commit SHA on every push, and additionally with `latest` only on pushes to main. Use the `GITHUB_TOKEN` for auth and the GHA cache backend for speedup. The build needs `packages: write` permission to push to GHCR."

Step 4: Pull and Run the Image on the Jetson (15 min)

Goal

Move from "image exists in GHCR" to "image runs on the Orin Nano." On the Jetson you'll authenticate with GHCR, pull the per-commit image you just built, run it with a volume mount so the TensorRT engine is cached across restarts, and confirm the engine actually produces detections. By the end of this step you'll have closed the loop: a `git push` from your laptop → a green CI run in the cloud → an ARM64 image in GHCR → that exact image running on your physical Jetson.

What success looks like

1. `docker pull` on the Jetson completes in 1–4 min (depends on campus network — first pull pulls all 8–10 GB, subsequent pulls only fetch changed layers)
2. `docker run` starts the container, the entrypoint says either `Compiling TensorRT engine` (first boot, takes 5–8 min) or `Reusing cached engine` (subsequent boots, ~5 s)
3. Once the engine is ready, `inference_node.py` produces detection output on stdout (exact format depends on your Lab 10 code)
4. A second `docker run` with the same volume mount starts in ~5 s instead of 5+ min — proves the cache works
5. A one-shot Python check from inside the running container loads the engine and runs inference on a synthetic frame

Step 4.1 — Pre-flight: what to check on the Jetson before pulling

Before pulling 10 GB you don't need to, confirm three things:

```
# 1. Free disk space — image + base layers take ~10 GB; you want ≥15 GB free
df -h /var/lib/docker | tail -1
# Look for at least 15G in the "Avail" column. If less, prune old images:
#   docker image prune -a      # removes everything not currently in use
#   docker system prune -af    # nuclear: also caches and stopped containers

# 2. nvidia container runtime is registered
docker info 2>/dev/null | grep -i "nvidia"
# Should print: Runtimes: io.containerd.runc.v2 nvidia runc
# If missing, follow JetPack docs for nvidia-container-toolkit install.

# 3. Mosquitto broker (only if your inference_node publishes — Lab 10 does)
systemctl status mosquitto --no-pager | head -5
# Should be active (running) on :1883.
```

Why we check disk first: the dustynv base image is ~7 GB, your application layers add ~1–2 GB, and Docker keeps both the previous image and the in-progress pull on disk during the operation, briefly using ~1.5x the final size. A pull that runs out of disk halfway through leaves Docker's overlay2 store in a half-mounted state that requires `docker system prune --volumes -af` to recover. Checking free space first is the cheapest insurance you'll buy today.

Step 4.2 — Log into GHCR from the Jetson

GHCR pulls of *public* images don't strictly need auth — anyone can pull a public image anonymously. We log in anyway for two reasons: (1) anonymous pulls are rate-limited (60/hour per IP, which a campus NAT shares across all 30 students), and (2) the same exact command works identically for private images later in HW6.

The cleanest path uses `gh` on the Jetson:

```
# If gh isn't installed yet:
sudo apt update && sudo apt install -y gh
gh auth login      # opens device-code flow; same login as your laptop

# Now hand the token to docker:
gh auth token | docker login ghcr.io -u <your-github-user> --password-stdin
```

Expected output:

```
Login Succeeded
```

Alternative if `gh` is unavailable: create a Personal Access Token at <https://github.com/settings/tokens> with `read:packages` scope, then:

```
echo "ghp_yourTokenHere" | docker login ghcr.io -u <your-github-user> --password-stdin
```

Don't paste the token directly into the command line — shell history will keep it. Either pipe from `gh auth token` (regenerates per session) or use an env var: `read -s GHCR_TOKEN; echo "$GHCR_TOKEN" | docker login ...`.

Step 4.3 — Pull the image

If you have the repo cloned on the Jetson, you can grab the SHA directly from git. Otherwise look it up in the GHCR web UI (Repo → Packages → `edgeai-lab12` → click any tag → copy the `sha-xxxxxxx` form).

```
# Option A: cloned the repo on the Jetson
cd ~/lab12 && git pull
SHA=$(git rev-parse --short HEAD)

# Option B: hardcode from the web UI
SHA=abc1234

# Save the full image ref for reuse
GHCR_USER=<your-github-user>
IMAGE="ghcr.io/${GHCR_USER}/edgeai-lab12:sha-${SHA}"
echo "$IMAGE"

# Pull — first time takes 1-4 min on a good campus link
docker pull "$IMAGE"
```

Expected output (last few lines):

```
sha-abc1234: Pulling from your-user/edgeai-lab12
71d6da26b0fb: Already exists
4f4fb700ef54: Pull complete
...
e5f0c8d1a2b3: Pull complete
Digest: sha256:9f8e7d6c5b4a392817263544fedcba0011223344556677889900aabbccddeeff
Status: Downloaded newer image for ghcr.io/.../edgeai-lab12:sha-abc1234
```

Verify locally:

```
docker images | grep edgeai-lab12
```

You should see something like:

Why the image is so big: the dustynv/pytorch base image carries CUDA 12.6 + cuDNN + TensorRT 10.3 + PyTorch 2.7 + torchvision + numpy with Jetson-specific aarch64 wheels. That's ~7 GB before your code is added. Multi-stage builds (HW5 covered this in depth) can knock it down to ~2 GB by stripping the dev toolchain, but for Lab 12 the single-stage size is fine — pulling once-per-commit is acceptable.

Step 4.4 — Run with engine cache volume + test video

Two volume mounts:

- **lab12-models** — named volume so the compiled `.engine` survives restarts. Without this, every restart rebuilds the engine from scratch, which is the 5–8 min you really want to amortize.
- **test_video.mp4** — `inference_node.py` defaults to `--source /opt/data/test_video.mp4`, but we deliberately don't bake test data into the image (production images carry code, not datasets). Mount your Lab 10 test clip at the path the script expects.

```
# 1. Engine cache volume (survives container restarts)
docker volume create lab12-models

# 2. Locate your test video — typically a deliverable from Lab 10.
#    Adjust the path on the left of the colon if yours lives elsewhere.
TEST_VIDEO=~/lab10/test_video.mp4
ls -lh "$TEST_VIDEO" # confirm it exists before mounting

# 3. Foreground run with --rm (auto-removes container on exit) and -it
#    (so Ctrl-C reaches Python via the entrypoint's exec).
docker run --rm -it \
  --runtime nvidia \
  --network host \
  -v lab12-models:/opt/models \
  -v "$TEST_VIDEO":/opt/data/test_video.mp4:ro \
  "$IMAGE"
```

Why `:ro`? The container only ever reads the test clip — never writes to it. Read-only bind mounts make that intent explicit and protect your host file from any accidental write inside the container.

Don't have a test video handy? Any short H.264 MP4 works as a stand-in (10+ s is enough). If you genuinely have nothing, override the source argument to use the IMX219

camera by replacing the default CMD on the docker run line — append the full python invocation, not just the flag:

```
docker run --rm -it --runtime nvidia -v lab12-models:/opt/models \
  "$IMAGE" python3 /app/inference_node.py --source 0
```

(Args appended after `<image>` *replace* the Dockerfile's `CMD` entirely — they don't append to it. So you have to repeat the full `python3 /app/inference_node.py` part, then add your overrides.)

On first boot you'll see this sequence (timestamps approximate):

```
[entrypoint] Compiling TensorRT engine (this takes 5-8 min on first boot)...
... (several minutes of TensorRT layer-fusion log lines, mostly silent)
[entrypoint] Engine compiled: 24M
[inference] Loading /opt/models/best.engine...
[inference] Connected to MQTT localhost:1883
[inference] frame 30, 38.1 fps, 2 dets
[inference] frame 60, 39.4 fps, 2 dets
...
```

If the inference node hangs at "Loading..." for more than a minute, the TRT version inside the container almost certainly mismatches the host driver — see [Troubleshooting](#) at the bottom of this step.

Stop the container with `Ctrl-C`. Because `entrypoint.sh` ends with `exec "$@"` (which runs the Dockerfile's `CMD` `python3 inference_node.py` directly), SIGINT goes to Python — which has handlers from `inference_node.py` for graceful shutdown. You should see:

```
^C
[inference] Received signal 2, shutting down...
[inference] Stopped. Processed 312 frames.
```

Re-run the same `docker run` command. This time the engine cache volume already has `best.engine`, so:

```
[entrypoint] Reusing cached engine: 24M
[inference] Loading /opt/models/best.engine...
[inference] Connected to MQTT localhost:1883
[inference] frame 30, 39.7 fps, 2 dets
```

Startup drops from ~8 min to ~5 s. **This is the moment the lab's CI/CD story pays off** — every future image rebuild produces the same `best.engine` binary location-on-disk via the volume, so deploys are near-instant after the first one.

Step 4.5 — Verify the detection pipeline three ways

You've proved the engine compiles and the inference loop runs. Now verify the engine actually does something useful.

Verification A — synthetic frame (no camera, no MQTT, fastest):

```
# In a second terminal on the Jetson, with the container still running:
CONTAINER=$(docker ps -q --filter ancestor="$IMAGE")
docker exec -it "$CONTAINER" python3 -c "
from ultralytics import YOLO
import numpy as np

m = YOLO('/opt/models/best.engine', task='detect')
img = (np.random.rand(320, 320, 3) * 255).astype('uint8')
r = m(img, verbose=False)
print(f'Detections returned: {len(r[0].boxes)} (zero is fine – random noise)')
print('PASS: engine inference works')
"
```

The point isn't the detection count (random noise produces unpredictable results) — it's that the engine loads, accepts an input, and returns a result without crashing.

Verification B — real test image:

```
# Copy a test image into the container (use any JPG you have)
docker cp ~/lab9/dataset/test/images/sample.jpg "$CONTAINER":/tmp/

docker exec -it "$CONTAINER" python3 -c "
from ultralytics import YOLO
m = YOLO('/opt/models/best.engine', task='detect')
r = m('/tmp/sample.jpg', verbose=False)
for b in r[0].boxes:
    print(f'  class={int(b.cls.item())} conf={float(b.conf.item()):.2f} xyxy={b.xyxy.tolist()[0]}')
"
```

Expect several boxes with confidences > 0.3. If you get zero boxes on a known-good image, the model file made it into the image but the engine compile took a wrong path — re-pull and re-run.

Verification C — MQTT round-trip:

```
# Subscribe in one terminal:
mosquitto_sub -h localhost -t 'jetson/vision/detections' -v | head -5
# Topic matches Lab 11's convention. If your inference_node.py still
# publishes to a different topic (e.g., the older 'edgeai/detections'),
# either update that node or change -t here to match what it publishes.

# The running container should already be publishing if your
# inference_node has an MQTT publisher wired up (Lab 10 default).
```

Expect JSON payloads streaming once per frame. This proves the full path works: container started → engine loaded → camera/video read → detection → MQTT publish.

Step 4.6 — Cache management (good to know)

The `lab12-models` volume persists across container restarts AND across `docker rm`. To start fresh (e.g., to test that your image's first-run compile path still works):

```
docker volume rm lab12-models      # delete the cache
docker volume create lab12-models  # recreate empty
# Next run will recompile, taking 5-8 min again.
```

To inspect what's in the cache:

```
docker run --rm -v lab12-models:/inspect alpine ls -la /inspect
# Should show: best.pt (6-10M for HW5-refined YOLOv8n), best.engine (~24M), best.onnx (sometimes)
```

Why we don't bake the engine into the image: the engine binary is built against a specific (driver, CUDA, TRT) triple. Bake it at image-build time and a different host with a different driver crashes on load. Compile at first run on the actual host, cache the result in a volume — this is the production pattern and it's what you'd do for a fleet of mixed-generation Jetsons.

How to Validate

Save as `validate_step4.sh` on the Jetson:

```
#!/usr/bin/env bash
# validate_step4.sh - verify the GHCR image pulls, runs, and exposes a
# working TRT engine + inference loop. Run this on the Orin Nano.
#
# Usage:
#   GHCR_USER=<your-github-user> SHA=<short-sha> [TEST_VIDEO=<path>] ./validate_step4.sh
#
# TEST_VIDEO defaults to ~/lab10/test_video.mp4. Override if your
# Lab 10 deliverable lives elsewhere.

set -e
echo "=== Step 4 validator: image runs on Jetson ==="

GHCR_USER="${GHCR_USER:?set GHCR_USER to your GitHub username}"
SHA="${SHA:?set SHA to the short git SHA you want to test (e.g. SHA=abc1234)}"
TEST_VIDEO="${TEST_VIDEO:-$HOME/lab10/test_video.mp4}"

IMAGE="ghcr.io/${GHCR_USER}/edgeai-lab12:sha-${SHA}"
```

```

# Pre-check: test video exists. inference_node.py defaults to
# /opt/data/test_video.mp4 inside the container; we mount the host
# file there so it can be opened.
[ -f "$TEST_VIDEO" ] || { echo "FAIL: test video not found at $TEST_VIDEO. Set TEST_VIDEO=<path>."
echo "PASS: test video found ($TEST_VIDEO)"

# 1. Image pulls
docker pull "$IMAGE" >/dev/null || { echo "FAIL: docker pull failed"; exit 1; }
echo "PASS: image pulled"

# 2. Architecture is arm64
ARCH=$(docker inspect "$IMAGE" --format='{{.Architecture}}')
[ "$ARCH" = "arm64" ] || { echo "FAIL: image arch is $ARCH, expected arm64"; exit 1; }
echo "PASS: arch = arm64"

# 3. Container starts, entrypoint compiles or reuses engine, then
# inference_node.py launches and starts publishing detections.
docker volume create lab12-models >/dev/null
docker rm -f lab12-validate >/dev/null 2>&1 || true
docker run --runtime nvidia --network host \
-v lab12-models:/opt/models \
-v "$TEST_VIDEO":/opt/data/test_video.mp4:ro \
--name lab12-validate -d "$IMAGE" >/dev/null \
|| { echo "FAIL: docker run failed"; exit 1; }

# Wait up to 12 min for the engine-ready signal (compile takes 5-8 min
# on first run; near-instant when the cache is warm).
ENGINE_READY=""
for i in {1..144}; do
    if docker logs lab12-validate 2>&1 | grep -qE "Reusing cached engine|Engine compiled"; then
        ENGINE_READY=yes
        echo "PASS: entrypoint compiled or reused engine"
        break
    fi
    sleep 5
done
[ -n "$ENGINE_READY" ] || {
    echo "FAIL: no engine ready signal in 12 min"
    docker logs lab12-validate | tail -20
    docker stop lab12-validate >/dev/null 2>&1 || true
    docker rm -f lab12-validate >/dev/null 2>&1 || true
    exit 1
}

# Wait up to 60 s for inference_node.py to start consuming the video
# (proves the cwd-leak / duplicate-CMD bugs in entrypoint.sh are gone
# and that the test-video volume mount works).
INFERENCE_OK=""
for i in {1..12}; do
    if docker logs lab12-validate 2>&1 | grep -qE "Running inference on|frames,.*FPS"; then
        INFERENCE_OK=yes
        echo "PASS: inference_node.py started and is processing the test video"
        break
    fi
    sleep 5
done
[ -n "$INFERENCE_OK" ] || {
    echo "FAIL: inference loop did not start within 60 s"
    docker logs lab12-validate | tail -20
}

```

```

docker stop lab12-validate >/dev/null 2>&1 || true
docker rm -f lab12-validate >/dev/null 2>&1 || true
exit 1
}

# Clean shutdown
docker stop lab12-validate >/dev/null 2>&1 || true
docker rm -f lab12-validate >/dev/null 2>&1 || true

echo "=== Step 4 PASS ==="

```

```

chmod +x validate_step4.sh
GHCR_USER=<your-user> SHA=<your-sha> ./validate_step4.sh

```

What the validator actually does, in 4 stages. (1) Pulls the image. (2) Confirms `arm64` architecture. (3) Starts the container with the test video mounted at `/opt/data/test_video.mp4` and waits for the entrypoint's `Engine compiled` or `Reusing cached engine` log line (timeout: 12 min — first compile takes 5–8 min). (4) Waits another 60 s for `inference_node.py` to log `Running inference on ...` or its first `frames, ... FPS` line, which proves the entrypoint cleanly exec'd the Dockerfile CMD and the volume mount works. Then it stops the container.

Step 4 Troubleshooting

Symptom	Likely cause	Fix
<code>docker pull</code> errors with <code>denied: requested access to the resource is denied</code>	GHCR package is private (default for new repos)	Repo Settings → Packages → <code>edgeai-lab12</code> → "Change package visibility" → Public, OR <code>docker login ghcr.io</code> with a token that has <code>read:packages</code>
<code>no matching manifest for linux/arm64 in the manifest list entries</code>	Build job didn't include <code>platforms: linux/arm64</code>	Re-check <code>.github/workflows/ci.yml</code> → build job → <code>platforms:</code> line. Push a fix and re-run.
<code>docker pull</code> extremely slow (>10 min) on lab Wi-Fi	Campus network is rate-limiting the dustynv layers	Pull on a wired connection if possible; once cached locally the layers are reused for subsequent pulls.
Container exits immediately with no logs	<code>entrypoint.sh</code> not executable in the image	Check the Dockerfile has <code>RUN chmod +x /app/entrypoint.sh</code> . Rebuild and re-push.

Symptom	Likely cause	Fix
Entrypoint says <code>ERROR: best.pt not found</code>	<code>best.pt</code> wasn't copied into the image (e.g. <code>.dockerignore</code> excluded it)	Check <code>.dockerignore</code> doesn't have <code>*.pt</code> ; rebuild.
Engine compile hangs forever (>15 min)	Out of GPU memory — another process owns it	<code>sudo systemctl stop nvargus-daemon</code> to free the camera-side allocator, kill any other CUDA processes (<code>nvidia-smi</code>), retry.
<code>[inference]</code> <code>Loading...</code> never returns	TRT version inside container doesn't match Jetson's TRT (driver mismatch)	The deferred-compile pattern usually prevents this, but if you accidentally baked an engine into the image, delete <code>lab12-models</code> volume and let it recompile.
Re-run after Ctrl-C says <code>Compiling</code> instead of <code>Reusing</code>	Volume mount missing or pointing at the wrong path	Confirm <code>-v lab12-models:/opt/models</code> is in the <code>docker run</code> line and the path matches what <code>entrypoint.sh</code> writes to.
<code>nvidia-container-runtime</code> not found	Toolkit not installed	<code>sudo apt install -y nvidia-container-toolkit</code> && <code>sudo systemctl restart docker</code>

Example AI Prompt

"I have an ARM64 Docker image at ghcr.io/myname/edgeai-lab12 that needs `--runtime nvidia` and a volume mount for cached TRT engines. Show me the docker pull + docker run commands, then write a bash validator that pulls, starts the container, waits for the entrypoint to either compile or reuse the engine, and exits 0 on success. Also include a troubleshooting checklist for the case where the container exits immediately with no logs."

Step 5: (Stretch) Self-Hosted Jetson Runner (10 min)

This step is optional for Lab 12. Steps 0–4 plus the TA sign-off cover the full pass criterion (a green CI pipeline producing an ARM64 image that runs on the Jetson). Step 5

is a stretch goal for teams who finish Step 4 with time left — it's also the first thing HW6 builds on, so it's worth doing now if you can. If you're tight on time, skip to Step 6.

Goal

Register your Orin Nano as a self-hosted GitHub Actions runner so a future job can build and test with **real GPU access** — no QEMU, no deferred-engine workaround. By the end of this step:

1. Your Jetson runner appears under **Settings** → **Actions** → **Runners** with status **Idle** (green dot)
2. The CI workflow gains a fourth job, `Smoke test on Jetson`, with `runs-on: [self-hosted, linux, arm64, jetson]`
3. On a `git push` from your laptop, the build job runs in GitHub's cloud → `jetson-smoke` job runs **on your physical Jetson** → both go green
4. `validate_step5.sh` confirms the runner is online and the latest workflow's `jetson-smoke` job succeeded

Why this matters. QEMU-emulated builds work but they're slow (30–50 min). A native Jetson runner cuts the build to ~6 min AND lets jobs talk to a real GPU — meaning the deferred-engine workaround in Step 2 becomes optional. For HW6, where the rubric requires an integration test that pulls the freshly-built image and does a real inference round-trip, a self-hosted runner isn't optional; it's the only path.

Step 5.1 — Download and register the runner

On the Jetson:

```
mkdir -p ~/actions-runner && cd ~/actions-runner

# Pin to a specific version so the lab is reproducible.
RUNNER_VER=2.319.1
curl -O -L "https://github.com/actions/runner/releases/download/v${RUNNER_VER}/actions-runner-linux-arm64-${RUNNER_VER}.tar.gz"
tar xzf actions-runner-linux-arm64-${RUNNER_VER}.tar.gz

# Get a one-time registration token from the GitHub UI:
# Repo → Settings → Actions → Runners → New self-hosted runner → Linux ARM64
# (or via CLI:)
gh api -X POST "/repos/${gh repo view --json nameWithOwner --jq .nameWithOwner}/actions/runners/register" \
  --jq .token

# Plug that token into the configure command:
./config.sh --url "https://github.com/${gh repo view --json nameWithOwner --jq .nameWithOwner}" \
```



```
--token "<paste-token-here>" \  
--labels "self-hosted,linux,arm64,jetson,orin-nano" \  
--name "jetson-team-XX" \  
--unattended
```

Step 5.2 — Run as a service

```
sudo ./svc.sh install  
sudo ./svc.sh start  
sudo ./svc.sh status    # should print active (running)
```

Step 5.3 — Add a job that targets the Jetson

Add this block to `ci.yml` after the existing `build` job:

```
jetson-smoke:  
  name: Smoke test on Jetson  
  runs-on: [self-hosted, linux, arm64, jetson]  
  needs: build  
  steps:  
    - uses: actions/checkout@v5  
    - name: Pull and verify image on this Jetson  
      # IMPORTANT: build IMAGE inside the `run:` shell, NOT in `env:`.  
      # YAML `env:` values are static strings — `$(...)` substitution  
      # does not run there. Doing it inside `run:` ensures the short  
      # SHA is computed at job time.  
      #  
      # We override the Dockerfile's CMD with a tiny `python3 -c`  
      # engine-load check rather than running the full inference loop.  
      # The entrypoint still fires first (compiles or reuses the  
      # engine), then exec's our verification command. This keeps the  
      # smoke test deterministic and short, and avoids needing to  
      # mount a test video on the runner.  
      run: |  
        SHORT_SHA=$(echo "${{ github.sha }}" | cut -c1-7)  
        IMAGE="ghcr.io/${{ github.repository }}:sha-${SHORT_SHA}"  
        echo "Pulling $IMAGE"  
        echo "${{ secrets.GITHUB_TOKEN }}" | docker login ghcr.io -u ${{{ github.actor }}} --passw  
        docker pull "$IMAGE"  
        docker volume create lab12-models  
        docker run --rm --runtime nvidia -v lab12-models:/opt/models \  
          "$IMAGE" python3 -c "  
from ultralytics import YOLO  
m = YOLO('/opt/models/best.engine', task='detect')  
print('PASS: engine loads on Jetson hardware')  
"
```

Push, watch the Actions tab — you should see a fourth job, "Smoke test on Jetson", run on your registered runner with the `jetson` label.

Step 5.4 — Cleanup (after TA sign-off)

Once your TA has signed off on Step 5, deregister and remove the self-hosted runner so your Jetson isn't picking up jobs from this repo on every push (and so the next team using this Jetson starts from a clean state). Run on the Jetson:

```
cd ~/actions-runner

# 1. Stop the systemd service
sudo ./svc.sh stop

# 2. Uninstall the systemd unit
sudo ./svc.sh uninstall

# 3. Deregister from GitHub. Two ways:
#
#   a) From the GitHub UI: Settings → Actions → Runners → click your
#       runner → Remove. GitHub deregisters server-side immediately.
#       After this, skip step 4's `config.sh remove` (it would fail
#       because there's no runner to remove on the server).
#
#   b) From the CLI: get a removal token from the same Settings page
#       (the `./config.sh remove --token XXX` command shown when you
#       hover Remove), then run it locally:
#       ./config.sh remove --token <removal-token>
#       This deregisters AND wipes local credentials in one shot.
#       Removal tokens expire in 1 hour like registration tokens.

# 4. Wipe the local actions-runner directory
cd ~
rm -rf ~/actions-runner

# 5. Verify both sides are clean
ls -d ~/actions-runner 2>&1          # "No such file or directory"
gh api /repos/<your-user>/edgeai-lab12/actions/runners --jq '.runners | length'
# Should print 0
```

Why deregister at all? A registered self-hosted runner remains assignable to the repo's workflows even when the Jetson is offline — GitHub queues jobs and will dispatch them the moment your runner reconnects (which happens automatically if `svc.sh` is still installed). Leaving it registered means a future `git push` from anyone with write access to the repo could trigger work on your Jetson without you noticing. Deregister when the lab is over.

How to Validate

Save as `validate_step5.sh`:

```

#!/usr/bin/env bash
# validate_step5.sh – verify a self-hosted Jetson runner is online and
# the jetson-smoke job ran on it.
#
# Usage:
#   REPO=<owner>/<repo> ./validate_step5.sh
#
# Without REPO, falls back to `gh repo view` (which only works when
# run from inside a clone of the GitHub repo). The env-var override
# lets you run this from anywhere – typical case is running from
# /home/jetson/project/coding/lab12/ (the TA reference) which is on
# a different remote than your GitHub fork.

set -e
echo "=== Step 5 validator: self-hosted runner ==="

# Resolve the repo. Env var wins; otherwise try `gh repo view` from cwd.
if [ -n "${REPO:-}" ]; then
    echo "Using REPO=$REPO (env override)"
elif REPO=$(gh repo view --json nameWithOwner --jq .nameWithOwner 2>/dev/null); then
    echo "Using REPO=$REPO (from current git remote)"
else
    echo "FAIL: cannot determine repo. Set REPO=<owner>/<repo> or run from inside a github.com clone"
    exit 1
fi

# Sanity: gh is authenticated
gh auth status >/dev/null 2>&1 || { echo "FAIL: gh not authenticated. Run 'gh auth login'."; exit 1; }
echo "PASS: gh authenticated"

# 1. Runner is registered and online
ONLINE=$(gh api "/repos/$REPO/actions/runners" \
    --jq '.runners[] | select(.status=="online") | .name' | wc -l)
[ "$ONLINE" -ge 1 ] || { echo "FAIL: no online self-hosted runners on $REPO"; exit 1; }
echo "PASS: $ONLINE self-hosted runner(s) online"

# 2. Latest workflow run included a jetson-smoke job. We use the REST API
# directly (not `gh run list`) so the call works regardless of cwd.
RUN_ID=$(gh api "/repos/$REPO/actions/runs?per_page=1" --jq '.workflow_runs[0].id')
[ -n "$RUN_ID" ] && [ "$RUN_ID" != "null" ] \
    || { echo "FAIL: no workflow runs found on $REPO"; exit 1; }
JOB=$(gh api "/repos/$REPO/actions/runs/$RUN_ID/jobs" \
    --jq '.jobs[] | select(.name=="Smoke test on Jetson")')
[ -n "$JOB" ] || { echo "FAIL: no 'Smoke test on Jetson' job in latest run ($RUN_ID)"; exit 1; }
CONC=$(echo "$JOB" | jq -r .conclusion)
[ "$CONC" = "success" ] || { echo "FAIL: jetson-smoke conclusion=$CONC"; exit 1; }
echo "PASS: jetson-smoke ran green on self-hosted runner"

echo "=== Step 5 PASS ==="

```

```

chmod +x validate_step5.sh
# If you're inside your laptop's GitHub clone of edgeai-lab12, gh repo
# view picks up the remote automatically. From any other directory
# (e.g. on the Jetson), set REPO explicitly:
REPO=<your-user>/edgeai-lab12 ./validate_step5.sh

```

Example AI Prompt

"Walk me through registering a Jetson Orin Nano (linux/arm64) as a self-hosted GitHub Actions runner for my repo. Include the curl download, the config.sh invocation with custom labels including 'jetson', running it as a systemd service, and then a YAML snippet for a job that uses runs-on: [self-hosted, jetson] to do a docker pull + docker run smoke test on the device."

Step 6: TA Sign-Off (time remaining)

Goal

Walk the TA through every artifact you produced today — the public repo, the green workflow runs, the GHCR image, and the on-device pull (plus the self-hosted runner if you did Step 5 stretch) — then run each `validate_stepN.sh` to demonstrate the end-to-end CI pipeline works.

Instructions

Show the TA, in this order:

1. **Repo home page** — public badge visible, README rendered.
2. **Actions tab** — at least three green workflow runs visible. Click into the most recent one and show the three required jobs (`Ruff lint`, `Pytest smoke`, `Build & push ARM64 image`) green. If Step 5 stretch was completed, also show the fourth `Smoke test on Jetson` job green.
3. **Packages tab** — `edgeai-lab12` listed, click in, show the `linux/arm64` manifest with both `sha-<...>` and `latest` tags.
4. **On the Jetson terminal** — run `docker run --rm --runtime nvidia -v lab12-models:/opt/models -v ~/lab10/test_video.mp4:/opt/data/test_video.mp4:ro ghcr.io/<user>/edgeai-lab12:latest` and let the TA see either `Reusing cached engine` (if the Step 4 cache is warm) or the full compile + the inference loop processing frames. `Ctrl-C` to stop.
5. **Settings** → **Actions** → **Runners** — show the Jetson registered as `Idle` or `Active`.

Then run the validators in order and show the TA the `PASS` lines:

```
cd ~/lab12
./validate_step0.sh      # repo + gh + files (7 PASS - 3 fixed + 4 file checks; +2 if PDM done)
./validate_step1.sh      # workflow green (1 PASS)
./validate_step2.sh      # Dockerfile refactored (4 PASS)
./validate_step3.sh      # ARM64 image in GHCR (3 PASS)
GHCR_USER=<user> SHA=<sha> TEST_VIDEO=<path-to-test_video.mp4> \
```

```
./validate_step4.sh      # image runs on Jetson (5 PASS)
# Stretch only — skip if you didn't do Step 5:
# REPO=<user>/edgeai-lab12 ./validate_step5.sh  # runner online + jetson-smoke green (3 PASS)
```

How to Validate

This is the validation — every prior step's `validate_stepN.sh` printing `PASS` plus the artifact tour above. The TA mirrors the `## TA Validation Steps` section at the end of this document.

Example AI Prompt

"My TA sign-off for Lab 12 is in 5 minutes. Help me write a 90-second narration that walks through (a) my GitHub repo's Actions tab and what each job did, (b) my GHCR Packages tab and why the manifest shows `linux/arm64` even though the build ran on a GitHub-hosted x86 runner, and (c) my Jetson registered as a self-hosted runner. Keep it in plain English — the TA cares about understanding the build/push/pull pipeline, not the YAML."

Submission Checklist

Before asking the TA to validate, work through each section below. **Tick every box** — anything left unchecked is something the TA will catch.

Files in `~/lab12/` (committed to the public GitHub repo)

Build artifacts (the application image's contents):

- ☐ `Dockerfile.ci` — no build-time GPU work (engine compile moved to `entrypoint.sh`)
- ☐ `entrypoint.sh` — executable (`chmod +x`), with the deferred engine-compile logic
- ☐ `inference_node.py`, `requirements.txt` — copied from your Lab 10 source location (`~/lab10/`, `~/project/coding/lab10/`, etc. — wherever your team keeps it)
- ☐ `best.pt` — `scp`'d from the Jetson (HW5's ~110-epoch refined weights preferred; Lab 9 baseline as fallback)

Project config (lints, tests, packaging):

- ☐ `pyproject.toml` — at minimum `[tool.ruff]` + `[tool.pytest.ini_options]` blocks (Step 1.2). Teams who did Step 0.4 PDM setup also have `[project]`, `[dependency-groups].dev`, `[tool.pdm.resolution]`, plus a committed `pdm.lock`.
- ☐ `tests/test_smoke.py` — four smoke-test functions (`test_best_pt_exists`, `test_requirements_pinned`, `test_dockerfile_uses_arm64_base`,

`test_required_files`); the parametrized `test_required_files` runs once each for `inference_node.py` / `best.pt` / `requirements.txt`, so pytest reports **6 collected items**

CI / validators:

- [] `.github/workflows/ci.yml` — three required jobs (`lint`, `test`, `build`); fourth `jetson-smoke` job only if Step 5 stretch was completed
- [] `validate_step0.sh` ... `validate_step4.sh` — five required validators; `validate_step5.sh` only if Step 5 stretch was completed

Validators (every one PASS)

- [] `./validate_step0.sh` — **7 PASS** (gh authenticated + origin on github.com + repo public + 4 required files; +2 more if you did Step 0.4 PDM and committed `pyproject.toml` + `pdm.lock`)
- [] `./validate_step1.sh` — **1 PASS** (latest workflow run on `main` is green within 3 min)
- [] `./validate_step2.sh` — **4/4 PASS** (entrypoint.sh + shebang + ENTRYPOINT line + no build-time engine compile)
- [] `./validate_step3.sh` — **3/3 PASS** (build workflow green + GHCR has `sha-<...>` tag + manifest reports arm64)
- [] `GHCR_USER=<user> SHA=<sha> TEST_VIDEO=<path> ./validate_step4.sh` — **5/5 PASS** (test video found + image pulls + arch=arm64 + engine ready + inference_node.py started processing the video)
- [] (*Stretch only*) `REP0=<user>/edgeai-lab12 ./validate_step5.sh` — **3/3 PASS** (gh authenticated + self-hosted runner online + `jetson-smoke` job ran green on it)

Live infrastructure (visible in the GitHub UI)

- [] **Public** GitHub repo at `github.com/<user>/edgeai-lab12`
 - [] At least one green workflow run on `main` (Actions tab)
 - [] `ghcr.io/<user>/edgeai-lab12` (Packages tab) with a `sha-<...>` arm64 tag and a `latest` tag
 - [] On the Jetson: `docker run` of the GHCR image works, engine compiles on first run and reuses cache on subsequent runs
 - [] (*Stretch only*) **Settings** → **Actions** → **Runners** — Jetson registered with status `Idle` or `Active`
-

TA Validation Steps

Each step is validated by a single command. Demonstrate each to the TA in order.

Step 0: Repo Plumbing (laptop)

```
./validate_step0.sh
```

TA checks: 7 PASS lines + Step 0 PASS (3 fixed checks + 4 file-presence iterations; +2 if PDM was set up in Step 0.4).

Step 1: Workflow Green (laptop)

```
./validate_step1.sh
```

TA checks: workflow conclusion = success + Step 1 PASS.

Step 2: Dockerfile Refactored (laptop)

```
./validate_step2.sh
```

TA checks: Four PASS lines + Step 2 PASS. TA also opens `Dockerfile.ci` and confirms no RUN line contains `format=engine`.

Step 3: ARM64 Image in GHCR (laptop)

```
./validate_step3.sh
```

TA checks: workflow completed successfully, GHCR has tag `sha-<...>`, manifest reports `arm64` + Step 3 PASS. (If `docker manifest inspect` is not available locally, TA visually confirms the `arm64` manifest in the GHCR web UI.)

Step 4: Image Runs on Jetson (Jetson)

```
GHCR_USER=<user> SHA=<sha> TEST_VIDEO=<path-to-test_video.mp4> ./validate_step4.sh
```

TA checks: 5 PASS lines (test video found, image pulled, arch = arm64, entrypoint compiled or reused engine, `inference_node.py` started and is processing the test video) + Step 4 PASS.

Step 5: (Stretch) Self-Hosted Jetson Runner (Jetson + laptop)

```
REPO=<user>/edgeai-lab12 ./validate_step5.sh
```

TA checks (only if attempted): 3 PASS lines (gh authenticated , N self-hosted runner(s) online , jetson-smoke ran green on self-hosted runner) + Step 5 PASS . Also opens **Settings** → **Actions** → **Runners** in the repo and confirms the Jetson is registered with status Idle or Active . Skipping this step does not affect the Lab 12 pass mark.

Step 6: TA Sign-Off

Confirm all required validate_step*.sh validators (Step 0 through Step 4) printed PASS ; if Step 5 stretch was attempted, also confirm validate_step5.sh PASS. Mark Lab 12 complete.

Troubleshooting

This is the cross-cutting / catch-all table — for issues specific to a single step, the per-step Troubleshooting block under that step is more targeted.

Authentication & access

Issue	Solution
gh: command not found	sudo apt install gh then gh auth login
gh auth status says "not logged in"	gh auth login and pick HTTPS + login via browser
gh works on the laptop but not the Jetson	They have separate auth state; run gh auth login on the Jetson too
error: gh auth token returns nothing	Token expired (90 days for SSO orgs). gh auth refresh

Git & repo

Issue	Solution
Push rejected: file too large (>100 MB)	git lfs install && git lfs track "*.pt" && git add .gitattributes , re-commit, push

Issue	Solution
Push rejected by branch protection on first push	You created the protection rule before pushing the initial commit; temporarily disable, push, re-enable
<code>git remote -v</code> shows no <code>origin</code>	<code>gh repo create --source=. --remote=origin --public --push</code>
Force-push to feature branch fails	<code>gh pr close && git push --delete origin <branch></code> then push fresh

CI / GitHub Actions

Issue	Solution
Workflow doesn't trigger after push	YAML lives at <code>.github/workflows/ci.yml</code> (typo in path is a common cause). Confirm with <code>find .github -type f</code>
Build job stuck > 60 min	QEMU is slow; confirm <code>cache-from: type=gha</code> is set. If first build, this is normal — wait
<code>Error: buildx failed with: ... unauthorized: authentication required</code>	Workflow missing <code>permissions: packages: write</code> block
Workflow shows <code>startup_failure</code>	YAML syntax error. Validate with <code>yamllint .github/workflows/ci.yml</code>
Cache always misses	The cache key includes the runner architecture; cross-arch cache is per-platform — expected behavior
Free-tier quota warning email	Switch repo visibility to Public (Settings → General → Danger Zone → Change visibility) — free unlimited Actions for public repos

GHCR & images

Issue	Solution
GHCR image not pullable from Jetson (<code>denied</code> error)	Package is private by default; in repo Settings → Packages → <code>edgeai-lab12</code> → "Change visibility" → Public, OR <code>docker login ghcr.io</code> with a token that has <code>read:packages</code>

Issue	Solution
<code>docker pull</code> errors with no matching manifest for <code>linux/arm64</code>	Build job's <code>platforms: linux/arm64</code> line missing or wrong; check the workflow file
GHCR shows two packages: <code>edgeai-lab12</code> and <code>edgeai-lab12/edgeai-lab12</code>	You set <code>images:</code> to <code>ghcr.io/\${{ github.repository }}/edgeai-lab12</code> instead of the un-doubled <code>ghcr.io/\${{ github.repository }}</code> (Step 3.2). Fix the YAML, push again — the new tag will land at the un-doubled path. The doubled package can be deleted from the GHCR web UI
Image size >12 GB	The single-stage build keeps the <code>dustynv</code> dev toolchain. HW5 covered the multi-stage path that drops to ~2 GB; for Lab 12 the larger size is acceptable

Jetson runtime

Issue	Solution
Container exits immediately on Jetson	<code>docker logs <container></code> — usually <code>entrypoint.sh</code> permissions (<code>chmod +x</code> not done) or <code>best.pt</code> not copied into the image
First-run engine compile hangs (>15 min)	Out of GPU memory; close other Jetson workloads (<code>sudo systemctl stop nvargus-daemon</code>), retry. Also check <code>nvidia-smi</code> for orphaned processes
Engine recompiles every restart	Volume mount missing. Add <code>-v lab12-models:/opt/models</code> to <code>docker run</code>
[inference] Loading... hangs forever	TRT version mismatch — delete the volume (<code>docker volume rm lab12-models</code> && <code>docker volume create lab12-models</code>) and let it recompile fresh
<code>nvidia-container-runtime</code> not found	<code>sudo apt install -y nvidia-container-toolkit</code> && <code>sudo systemctl restart docker</code>

Self-hosted runner

Issue	Solution
Self-hosted runner shows <code>offline</code>	<code>sudo ./svc.sh status</code> from <code>~/actions-runner</code> ; restart with <code>sudo ./svc.sh restart</code>

Issue	Solution
Runner accepts job but it never starts	Labels mismatch between job's <code>runs-on:</code> and runner's registered labels. <code>gh api /repos/.../actions/runners</code> shows the labels
Runner job fails with <code>permission denied</code> for docker	Add the runner user to docker group: <code>sudo usermod -aG docker \$(whoami)</code> and re-login
Two teams' runners both pick up jobs	Add a unique label per team: <code>--labels "self-hosted,linux,arm64,jetson,team-XX"</code> and target with <code>runs-on: [self-hosted, team-XX]</code>

Key Concepts from Today's Lecture

Concept	Where It Came From	How You Used It Today
GitHub Actions workflow YAML	Week 10 <code>github-actions</code> slide	Wrote <code>.github/workflows/ci.yml</code> with three jobs
CI vs CD	Week 10 <code>cicd-overview</code> slide	Lint+test = CI; build+push to GHCR = the start of CD
Multi-arch Docker builds	Week 10 <code>docker-on-jetson</code> slide	Used <code>docker buildx</code> + QEMU to build <code>linux/arm64</code> from x86 runner
Container registries	Week 10 <code>dockerfile-best-practices</code> slide	Pushed to GHCR with commit SHA + <code>latest</code> tags
Self-hosted runners	This week's lecture, <code>cicd-tools-comparison</code> slide	Registered the Jetson with <code>jetson</code> label, used <code>runs-on: [self-hosted, jetson]</code>
Build caching	Week 10 <code>dockerfile-optimization</code> slide	<code>cache-from: type=gha</code> reuses pip layer across builds

What's Next

Today's pipeline is the substrate everything below builds on. Here is how the skills you practiced today connect to upcoming weeks and the final project:

Where	Connection to Lab 12
HW6 — Production deploy + INT8 calibration	Extends today's <code>ci.yml</code> into a 5-job pipeline (<code>Ruff lint</code> , <code>Pytest + coverage</code> , <code>Bandit + pip-audit</code> , <code>Build & push ARM64 image</code> , <code>Integration test (Jetson)</code>). Adds branch protection on <code>main</code> (Step 0.4) with the required-status-check list. Wires INT8 calibration + benchmark automation across every power mode <code>/etc/nvpmode1.conf</code> reports.
HW6 — Coverage + accuracy gates	<code>pytest --cov-fail-under=90</code> makes CI fail when test coverage drops. An accuracy gate fails CI if INT8 mAP drops more than 2 pts vs FP16. Both run as required status checks today's pipeline already enforces the <i>shape</i> of.
HW6 — Self-hosted runner integration test	Parametrized integration test runs on the Jetson runner you registered in Step 5, sweeping each <code>nvpmode1</code> mode. Today's runner registration is the prerequisite.
HW6 — Adaptive power + rollback	Deploy job SSHes into the Jetson, sets <code>nvpmode1</code> , pulls the new image, restarts the compose stack. Adaptive power manager switches modes based on temperature and load. Rollback in <30 s. The GHCR pull pattern from Step 4 is the substrate.
Week 13 (Security & Privacy)	TLS for MQTT, JWT auth, face anonymization, FGSM adversarial robustness. Today's CI/CD pipeline is what would <i>enforce</i> those controls in HW6's bonus security-scanning job (Trivy image scan, Bandit Python SAST).
Final project — Fleet management	Today's one-Jetson workflow scales to a 10-Jetson fleet by adding labels per device (<code>jetson-team-1</code> , <code>jetson-team-2</code> , ...) and parametrizing <code>runs-on</code> : . The deploy job becomes a matrix over devices.

Heads-up for HW6: the test job will be renamed (`Pytest smoke` → `Pytest + coverage`) and three new jobs will be added (`Bandit + pip-audit` , `Integration test (Jetson)` , plus the deploy workflow). HW6 Step 0.4 sets up branch protection with the right required-status-check list at the start of the homework.

Great work — you've built a real CI/CD pipeline that catches bugs before they hit your Jetson, ships a multi-arch image to a public registry on every commit, and proves the image runs on real hardware. The rest of the course (and your career) will assume this baseline.